

ORIGINAL ARTICLE

Cost-effective economic dispatch in large-scale power systems using enhanced manta ray foraging optimization

S. R. Spea¹

Received: 24 September 2024 / Accepted: 8 February 2025 / Published online: 3 April 2025

© The Author(s) 2025

Abstract

Economic dispatch (ED) is a critical optimization problem in power systems, challenged by real-world constraints such as prohibited operating zones (POZ), valve-point loading effects (VPL), and multi-fuel options (MFO). POZ are regions where generators cannot operate due to mechanical limitations; VPL introduces cost fluctuations caused by turbine valve operations; and MFO allows generators to switch between multiple fuel types, adding complexity to cost functions. To address these challenges, this study proposes four enhanced Manta Ray Foraging Optimization (MRFO) variants: opposition-based MRFO (OMRFO), quasi-oppositional MRFO (QMRFO), opposition-based generation jumping MRFO (JOMRFO), and quasi-oppositional generation jumping MRFO (JQMRFO). These variants leverage opposition-based learning (OBL), quasi-oppositional learning (QOBL), and a generation jumping mechanism to balance exploration and exploitation, overcoming limitations of standard MRFO, such as slow convergence and local optima entrapment. OBL and QOBL diversify the search by generating opposite or quasi-opposite solutions, expanding the search space, and avoiding stagnation. The jumping mechanism introduces probabilistic “jumps” to explore non-adjacent regions, enhancing exploration further. Exploitation is refined by retaining and improving the most promising solutions. The algorithms are tested on standard benchmark systems widely used in power systems literature, including the 10, 15, 40, 140, and 160-unit systems, ensuring comparability and reproducibility. Results show that JOMRFO outperforms MRFO and other state-of-the-art methods, achieving significant annual cost savings: \$3,730,344 on a 10-unit system, \$54,641,376 on a 40-unit system, and \$955,501,418 on a 140-unit system. These findings highlight the effectiveness of the proposed variants in improving optimization efficiency and reducing operational costs.

Keywords Economic dispatch · Opposition-based learning · Quasi-oppositional learning · Generation jumping · Manta ray foraging optimization · Power systems

Abbreviations

aPSO	Advanced particle swarm optimization
BBO	Biogeography-based optimization
BWO	Beluga whale optimizer
CBA	Chaotic bat algorithm
CCSO	Clustering cuckoo search optimization
CGA-MU	Classical genetic algorithm with multiple update
C-GRASP-SaDE	Continuous greedy randomized adaptive search procedure in combination with a self-adaptive DE approach



Ch-JAYA	Chaotic JAYA algorithm
CSA	Cuckoo search algorithm
DHS	Differential harmony search algorithm
DSPSO-TS	Distributed Sobol PSO and TSA
ED	Economic dispatch
ED-DE	Estimation of Distribution and Differential Evolution
EMA	Exchange market algorithm
EMA-SS	Exchange market algorithm with smart searching
EO	Equilibrium optimizer
EO-SCA	Hybrid equilibrium optimizer and sine–cosine algorithm
EPSO	Evolutionary particle swarm optimization
ESCSDO	Eagle-strategy supply–demand optimizer
ESNS	Enhanced social network search
FA	Firefly Algorithm
FOX	Fox optimizer
GA	Genetic algorithm
GA-API	GA with API (special class of ant colony optimization)
GABC	Gbest guided artificial bee colony algorithm
GBO	Gradient-based optimizer
GJO	Golden jackal optimization
GNDO	Generalized normal distribution optimization algorithm
GNDO	Generalized normal distribution optimization
GWO	Grey wolf optimization
haDEPSO	Hybridization of advanced differential evolution and PSO
HBB-BC	Hybrid big bang–big crunch optimization
HGS	Hunger games search
HHO	Hybrid Harris Hawks optimizer
HHO-A β HC	HHO with adaptive-hill-climbing optimizer
HS	Harmony search algorithm
HSSA	Hybrid Salp Swarm Algorithm
IA_EDP	Immune algorithm for economic dispatch problem
IBWO	Enhanced beluga whale optimizer
IGA-MU	Improved genetic algorithm with multiple update
IODPSO-G	Multiple Strategies based Orthogonal Design PSO with global structure
IODPSO-L	Multiple Strategies based Orthogonal Design PSO with local structure
ISMA	Improved slime mould algorithm
Jaya-M	Jaya algorithm with multi-population
Jaya-SML	Jaya algorithm with self-adaptive multi-population and Lévy flights
JOMRFO	Opposition-based generation jumping MRFO
JQMRFO	Quasi-oppositional generation jumping MRFO
J_r	Jumping rate
JS	Jellyfish Search optimizer
KHA	Krill herd algorithm
KHA-IV	Krill herd algorithm with crossover and mutation operators
LWSO	Leader white shark optimizer
MABC	New artificial bee colony algorithm
MFO	Multi-fuel options
MPA	Marine predator algorithm

MPSO-GA	Modified particle swarm and genetic algorithm
MRFO	Manta Ray Foraging Optimization
MSGO	Modified social group optimization
MTVPSO	Modified time-varying PSO
NGO	Northern goshawk optimization
OBL	Opposition-based learning
OGWO	Oppositional based grey wolf optimization algorithm
OIWO	Oppositional invasive weed optimization
OKHA	Opposition-based krill herd algorithm
OMRFO	Opposition-based MRFO
ORCCRO	Oppositional real coded chemical reaction optimization
PARPSO	Penalty attractive and repulsive PSO
POZs	Prohibited operating zones
PPSO	Phasor particle swarm optimization
PSO	Particle swarm optimization
PSO-CGSA	PSO and chaotic gravitational search algorithm
PSOSIF	PSO with smart inertia factor
QMRFO	Quasi-oppositional MRFO
QOBL	Quasi-oppositional learning
QOPO	Quasi-oppositional search-based political optimizer
RUN	RUNge Kutta optimizer
SCA- β HHC	Sine cosine algorithm- β -hill climbing optimizer
SDE	Shuffled differential evolution
SDO	Supply-demand based optimization
SGO	Social group optimization
SMA	Slime mould algorithm
SNS	Social network search
SSA	Slap swarm algorithm
TLBO	Teaching learning-based optimization
TS	Tabu search algorithm
VPL	Valve-point loading effects
WSO	White shark optimizer

1 Introduction

Using computational intelligence techniques to handle optimization problems is a growing trend in power systems research. These techniques are valued for their ability to efficiently and accurately handle large and complex systems [1]. In the realm of power system operation and planning, economic dispatch (ED) is a critical optimization problem that seeks to minimize generation costs while meeting system load demands and adhering to various constraints [2].

Conventional optimization algorithms, such as the gradient method, linear programming, quadratic programming, and the Lagrangian relaxation algorithm, have served as foundational approaches for addressing the ED problem. However, these methods often struggle to handle the non-convex characteristics of practical ED scenarios, which arise from discontinuities and higher-order nonlinearities in modern generating units caused by

factors like valve-point loading effects (VPL), prohibited operating zones (POZ), and multi-fuel options (MFO) [3, 4].

Metaheuristic algorithms, known for their robustness in solving high-dimensional and non-convex problems, have emerged as powerful alternatives. These algorithms do not rely on gradient information, making them inherently more suitable for the complex and multimodal search spaces of ED problems. Studies employing benchmark test systems, widely recognized for their diverse characteristics and real-world complexity, have contributed to evaluating and validating these algorithms. For example, the leader white shark optimizer (LWSO) [5] was tested on standard 6-, 10-, 11-, and 40-unit systems, and an enhanced social network search (ESNS) algorithm was validated using standard 11-, 15-, 40-, and 110-unit systems in [6]. The oppositional-based MRFO algorithm (OMRFO) addressed various ED challenges and was evaluated on 5-, 6-, 24-, and 40-unit systems [7]. Additionally, a modified cheetah optimizer (MCO) that integrates opposition-based learning (OBL) and a dynamic adaptive weighting factor was implemented to address multi-objective ED problems characterized by non-smooth cost functions and ramp-rate limits. The MCO algorithm was tested on standard 3-, 13-, and 40-unit systems [8].

Other studies further validated their approaches using recognized test systems. For instance, the oppositional invasive weed optimization (OIWO) algorithm tackled large-scale ED problems by drawing inspiration from the colonizing behaviour of weed plants, enhanced by the quasi-opposite technique [9]. It was validated on standard 13-, 40-, 110-, 140-, and 160-unit test systems. Similarly, the quasi-oppositional search-based political optimizer (QOPO) addressed both single and multi-objective economic-emission dispatch problems, testing its effectiveness on standard 3-, 6-, 10-, 11-, 13-, and 40-unit test systems while considering VPL, transmission power losses, and generating unit constraints [10]. Both the oppositional-based krill herd algorithm (OKHA) [11] and quasi-oppositional grey wolf optimizer (GWO) [12] were utilized to address the ED problem incorporating OBL and QOBL, respectively, with a jumping rate (J_r) to facilitate escaping from local optima without increasing computational complexity; the OKHA was tested on standard test systems ranging from 6 to 140 units, while the GWO was employed to solve the ED problem for test systems comprising 3-, 5-, 6-, 38-, and 110-units. An additional literature review analysing relevant articles is summarized in Table 1, which outlines the optimization algorithms applied, descriptions of the standard test systems, practical constraints, and publication years.

The existing body of research on economic dispatch (ED) highlights significant progress but reveals critical gaps that warrant further investigation. While these algorithms offer compelling results, they often struggle with premature convergence and suboptimal solutions due to the complex search space of the economic dispatch

Table 1 Extensive review of relevant literature

Author [Ref. No.]	Optimization method	Standard test system description	Constraints				Year
			VPL	POZ	Power losses	MFO	
Secui et al. [13]	MSGO	10- and 40-unit	✓	×	✓	×	2024
Hoque et al. [14]	GNDO	3-, 6-, and 10-unit	×	×	×	×	2023
Hassan et al. [15]	ESCSDO	6-, 13-, 15, and 40-unit	✓	✓	✓	×	2023
Al-Betar et al. [16]	HHO	6-, 13-, 15-, 40-, and 140-unit	✓	✓	✓	×	2023
Al-Betar et al. [17]	SCA-βHC	3-, 6-, 13-, 15-, and 40-unit	✓	✓	✓	×	2023
Yu et al. [18]	CCSO	6-, 10-, 13-, 15-, and 40-unit	✓	✓	✓	✓	2020
Yu et al. [19]	Jaya-SML	6-, 10-, 13-, 15-, and 40-unit	✓	✓	✓	✓	2019
Adarsh et al. [20]	CBA	6-, 13-, 20-, 40- and 160-unit	✓	✓	✓	×	2016
Bhattacharjee et al. [21]	SSA	6-, 10-, 13-, 40- and 140-unit	✓	✓	✓	×	2022
Spea	Proposed MRFO and its modified variants	10-, 15-, 40-, 140- and 160-unit	✓	✓	✓	✓	

problem. Furthermore, a considerable portion of the literature focuses on ED problems in small and medium-scale test systems, neglecting large-scale systems and critical real-world constraints, creating gaps in understanding their scalability and practical applicability. Additionally, while oppositional-based learning (OBL) and its variants (QOBL, JOBL, JQOBL) have enhanced algorithm performance, a comprehensive comparative analysis of their effectiveness remains lacking.

This paper addresses these gaps by leveraging Manta Ray Foraging Optimization (MRFO) as a robust optimization tool [22]. Unlike many conventional and metaheuristic approaches, the parameter-free nature of MRFO simplifies implementation by eliminating the need for complex tuning processes. Its reliance on natural foraging behaviour aligns well with the dynamic search space of ED problems, where balancing exploration and exploitation is crucial. However, the standard MRFO algorithm, like many other metaheuristics, often suffers from premature convergence, local optima entrapment, and slower convergence rates, particularly in complex, large-scale systems [22–26]. These limitations can hinder its effectiveness when used directly in economic dispatch problems with high dimensionality, multimodality, and non-convexity.

To further enhance MRFO's suitability for ED problems, this study introduces variants incorporating opposition-based learning techniques (OBL, QOBL, JOBL, and JQOBL). These enhancements are particularly advantageous for ED problems because they address:

- **Exploration of Non-Convex Search Spaces:** Oppositional-based techniques improve the search process by generating oppositional/quasi-oppositional solutions, increasing the likelihood of escaping local optima—critical for navigating the discontinuities and non-convexities of ED.
- **Adaptability to Diverse System Scales:** Introducing the “jumping rate” mechanism in JOBL and JQOBL, allowing the algorithm to adaptively transition between solutions and their opposites. This dynamic mechanism fosters a more effective balance between global exploration and local refinement, crucial for maintaining solution quality across varying ED problem scales and constraints.
- **Computational Efficiency:** By improving convergence speed without sacrificing accuracy, the enhanced MRFO variants are particularly well-suited for real-time ED applications.

In comparison with other metaheuristics, MRFO-based approaches combine simplicity and adaptability, making them highly effective for addressing the practical challenges of ED problems. Specifically, integrating opposition-based learning techniques further strengthens MRFO's ability to escape local optima and achieve superior solution quality.

The contributions of the paper are as follows:

1. Addressing ED problems in large-scale test systems characterized by a substantial number of generation units.
2. Incorporating real-world constraints such as VPL, POZ, MFO, and transmission power losses for realistic ED problem modelling.
3. Enhancing MRFO performance using opposition-based learning (OBL, QOBL, JOBL, and JQOBL), balancing exploration and exploitation, improving convergence speed, and reducing local optima entrapment.
4. Providing a comparative analysis of opposition-based learning techniques to identify their relative strengths and limitations for ED problems.
5. Identifying the most effective oppositional-based technique for minimizing generation costs under varying system sizes and constraints.

The findings demonstrate that integrating opposition-based learning techniques into MRFO significantly enhances its performance, making it a robust and scalable solution for complex ED problems.

The remaining sections of the paper are organized as follows: Sect. 2 describes the mathematical model of the ED problem. Section 3 presents the MRFO algorithm and its enhanced variants. Section 4 details the implementation of the proposed algorithms. Section 5 summarizes the simulation results and discussions. Finally, Section 6 concludes the paper.

2 Problem formulation

The fundamental objective of the ED problem is to attain the most cost-effective loading of generating units, thereby minimizing fuel costs while fulfilling system load demand. The ED also seeks to ensure compliance with operational constraints such as transmission power losses and generator output limits.

2.1 Objective function

The different ED cost functions may be mathematically expressed as follows:

1. Traditional ED.

The quadratic cost function is mainly used in traditional ED. This function is defined as follows [1]:

$$F(P_g) = \sum_{n=1}^{N_g} F_n(P_{gn}) = \sum_{n=1}^{N_g} (a_n + b_n P_{gn} + c_n P_{gn}^2) \quad n = 1 : N_g \quad (1)$$

where $F(P_g)$ is the total cost function in \$/h, $F_n(P_{gn})$ is the cost function of the n th generation unit in \$/h, P_{gn} is the generator output power in MW, N_g is the total number of generation units, and a_n , b_n , and c_n are the cost coefficients of the n th generator.

2. Fuel cost function with valve-point loading effects.

The fuel-cost curve for thermal generating units often exhibits non-smooth behaviour characterized by sinusoidal ripples, which arise due to the valve-point loading (VPL) effect. This effect reflects the sequential opening of multiple valve stages in steam turbines as power output increases. At specific operating points, known as valve points, the sudden increases in steam flow led to abrupt changes in fuel consumption. These transitions introduce periodic fluctuations, resulting in a wave-like pattern on the fuel-cost curve (see Fig. 1) [2, 10]. To accurately represent these irregularities in optimization models, sinusoidal components are added to the quadratic cost function, as shown in Eq. (2) [13]. This adjustment captures the non-linear behaviour introduced by VPL, improving the realism and effectiveness of economic load dispatch optimization.

$$F(P_g) = \sum_{n=1}^{N_g} \left(a_n + b_n P_{gn} + c_n P_{gn}^2 + \left| d_n \sin \left\{ e_n (P_{gn}^{min} - P_{gn}) \right\} \right| \right) \quad (2)$$

Fig. 1 Cost function curve with and without valve-point loading effects

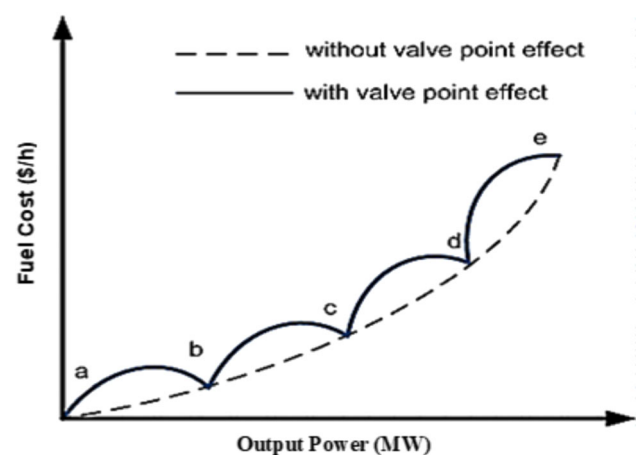
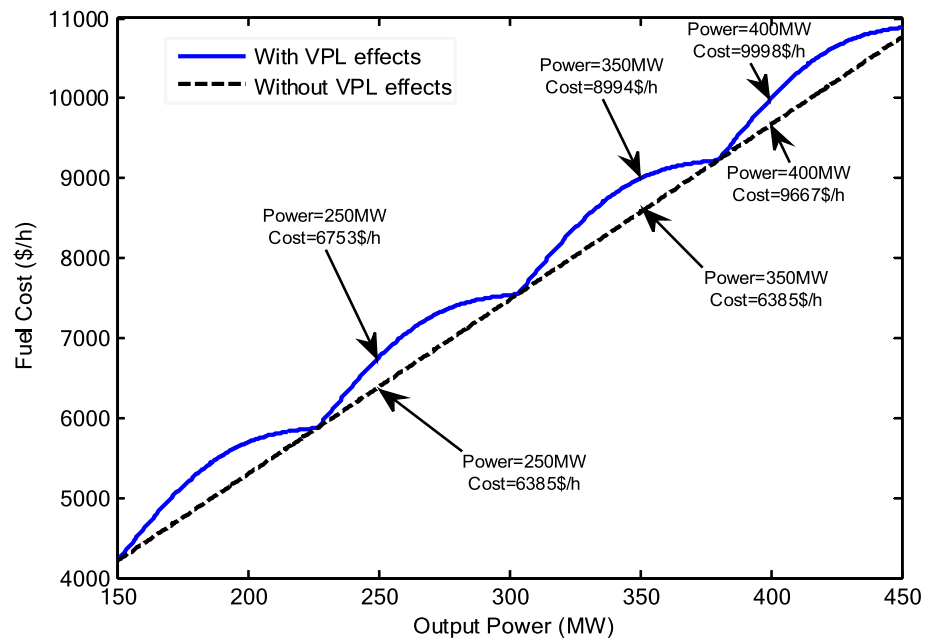


Fig. 2 Impact of valve-point loading effects on fuel-cost curve for a 450 MW steam turbine generator



where d_n and e_n are the coefficients of the n th generation unit reflecting VPL, and P_{gn}^{min} is the minimum generated output power of the n th generator.

Figure 2 illustrates the impact of VPL on the cost curve for a 450 MW steam turbine generator using the following parameters: $a = 0.00043$, $b = 21.60$, $c = 958.20$, $d = 450$, $e = 0.041$, and $P_{gn}^{min} = 150$ [27]. The dashed line represents the curve without VPL effects, which appears linear due to the small cost coefficient. The solid line incorporates sinusoidal components to account for VPL effects. The inclusion of VPL effects significantly increases fuel costs. For example, at 250 MW, the cost is 6753 \$/h, a 5.76% increase compared to the cost without VPL. At 350 MW, the cost with VPL is 8994 \$/h, representing a 4.94% increase. At 400 MW, the cost with VPL is 9998 \$/MWh, which is approximately 3.42% higher. These findings underscore the importance of including VPL effects in economic dispatch models to obtain accurate cost estimations.

3. Fuel cost function with multiple fuel options.

Thermal power stations often utilize generation units that can be fuelled by different fuels. To accurately represent the cost associated with these fuel types, the fuel cost objective function is expressed as a piecewise quadratic function, as shown in (3), capturing the impact of switching between fuels [18]. The ED with MFO seeks to identify the most appropriate fuel for each generation unit to minimize total fuel costs. The effect of MFO on the fuel-cost curve is shown in Fig. 3 [28, 29].

$$F_n(P_{gn}) = \begin{cases} a_{n1} + b_{n1}P_{gn} + c_{n1}P_{gn}^2 & P_{gn}^{min} \leq P_{gn} \leq P_{gn1} \text{ for fuel 1} \\ a_{n2} + b_{n2}P_{gn} + c_{n2}P_{gn}^2 & P_{gn1} \leq P_{gn} \leq P_{gn2} \text{ for fuel 2} \\ \vdots & \vdots \\ a_{nL} + b_{nL}P_{gn} + c_{nL}P_{gn}^2 & P_{gnL-1} \leq P_{gn} \leq P_{gn}^{max} \text{ for fuel } L \end{cases} \quad (3)$$

where a_{nL} , b_{nL} , and c_{nL} are the cost coefficients of the n th generator for the L th fuel type.

Modelling the fuel cost function accurately requires consideration of both the non-linear VPL and the discrete cost changes associated with MFO. Equation (4) [19] presents this combined model, which incorporates both

Fig. 3 Impact of MFO on cost function curve

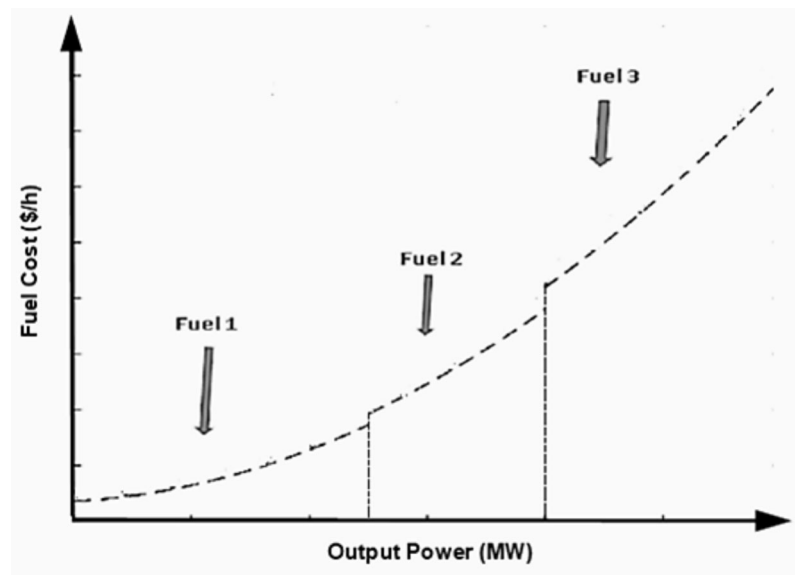
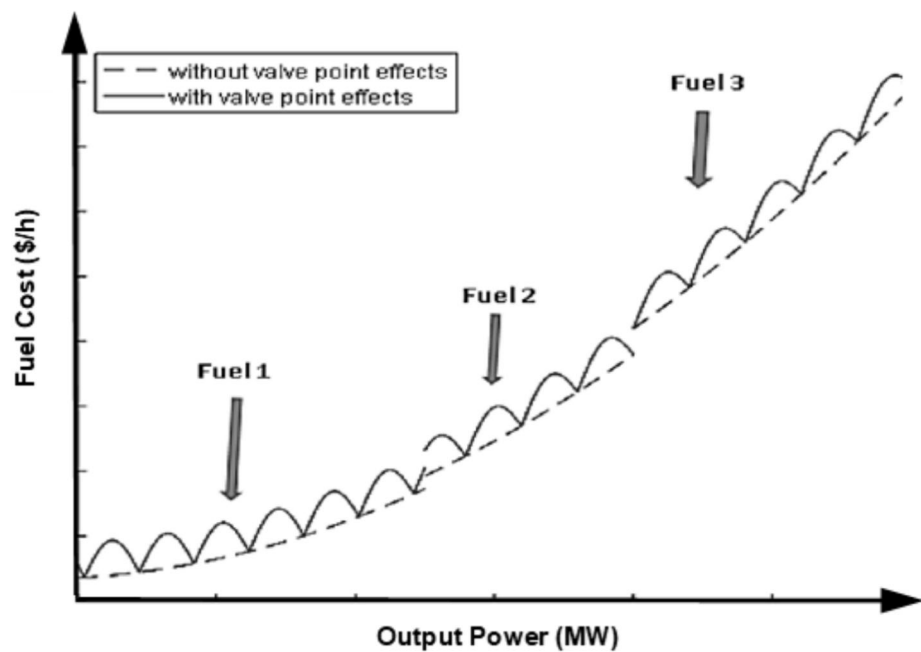


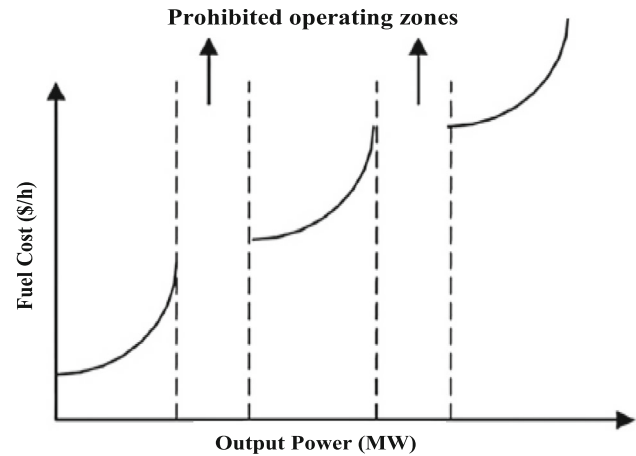
Fig. 4 Impact of MFO and VPL effects on cost function curve



VPL's sinusoidal ripples and the piecewise nature of MFO fuel costs. Figure 4 illustrates the fuel-cost characteristic of a typical generating unit under this combined model [29].

$$F_n(P_{gn}) = \begin{cases} a_{n1} + b_{n1}P_{gn} + c_{n1}P_{gn}^2 + |d_{n1} \sin\{e_{n1}(P_{gn}^{\min} - P_{gn})\}| & P_{gn}^{\min} \leq P_{gn} \leq P_{gn1} \text{ for fuel 1} \\ a_{n2} + b_{n2}P_{gn} + c_{n2}P_{gn}^2 + |d_{n2} \sin\{e_{n2}(P_{gn}^{\min} - P_{gn})\}| & P_{gn1} \leq P_{gn} \leq P_{gn2} \text{ for fuel 2} \\ \vdots & \vdots \\ a_{nL} + b_{nL}P_{gn} + c_{nL}P_{gn}^2 + |d_{nL} \sin\{e_{nL}(P_{gn}^{\min} - P_{gn})\}| & P_{gnL-1} \leq P_{gn} \leq P_{gn}^{\max} \text{ for fuel L} \end{cases} \quad (4)$$

Fig. 5 Fuel-cost curve for a thermal generating unit with POZs



2.2 Constraints

The practical ED problem should be addressed based on the following constraints:

(1) Power balance constraint: The total power produced must be equal to the total load demand (P_D) as well as the transmission power losses (P_{loss}) [2]. Therefore,

$$\sum_{n=1}^{N_g} P_{gn} = P_D + P_{loss} \quad (5)$$

The P_{loss} may be determined by employing Kron's loss formula (B -coefficient) as follows [3]:

$$P_{loss} = \sum_{m=1}^{N_g} \sum_{n=1}^{N_g} P_{gm} B_{mn} P_{gn} + \sum_{m=1}^{N_g} P_{gm} B_{om} + B_{oo} \quad (6)$$

where B_{mn} , B_{om} , and B_{oo} are the loss coefficients.

(2) Generation limits constraint: The power output from each generation unit has to be within the predefined limits as follows [4]:

$$P_{gn}^{min} \leq P_{gn} \leq P_{gn}^{max} \quad n = 1 : N_g \quad (7)$$

(3) Prohibited operating zone constraint: To maintain reliable operation and avoid potential instabilities, thermal generating units often have prohibited operating zones (POZs) where operation is unfavourable due to the physical limitations of machine components. These POZs create discontinuities in the fuel-cost curve, as illustrated in Fig. 5. The mathematical representation of this non-continuous fuel-cost function is given by [15]:

$$\begin{aligned} P_{gn}^{min} &\leq P_{gn} \leq P_{gn,m}^{min} \\ P_{gn,m}^{max} &\leq P_{gn} \leq P_{gn,m+1}^{min} \quad m = 1 : PZ_n \\ P_{gn,PZ_n}^{max} &\leq P_{gn} \leq P_{gn}^{max} \end{aligned} \quad (8)$$

where $P_{gn,m}^{min}$ and $P_{gn,m}^{max}$ are minimum and maximum limits of the m th prohibited zone of unit n , and PZ_n is the total number of POZ of unit n .

3 Proposed algorithms

3.1 Conventional manta ray foraging optimization algorithm (MRFO) overview

Introduced by Zhao et al. in 2020 [22], the conventional MRFO algorithm models the foraging behaviours of manta rays, including chain, cyclone, and somersault foraging, depicted in Fig. 6. These foraging behaviours can be mathematically described as follows:

1. Chain foraging strategy: In this strategy, the manta rays are arranged in a line to catch the maximum quantity of food in their gills. Every member of the chain, except the first, moves to the food and the one ahead. Consequently, each member's position is updated in each iteration according to the best solution that has been achieved so far, as well as the solution ahead of it [7, 22].

$$x_i^{t+1} = \begin{cases} x_{i,d}^t + r \cdot (x_{best,d}^t - x_{i,d}^t) + \alpha \cdot (x_{best,d}^t - x_{i,d}^t) & i = 1 \\ x_{i,d}^t + r \cdot (x_{i-1,d}^t - x_{i,d}^t) + \alpha \cdot (x_{best,d}^t - x_{i,d}^t) & i = 2, \dots, N \end{cases} \quad (9)$$

$$\alpha = 2 \cdot r \cdot \sqrt{|\log(r)|} \quad (10)$$

where $x_{i,d}^t$ is the i th position of the individual at time t in the d th dimension, r is a random number between 0 and 1, α is a weight coefficient, $x_{best,d}^t$ is the plankton prey having the maximum concentration, and N is the total number of manta rays.

2. Cyclone foraging strategy: In this strategy, the manta rays accompany in a spiral way to cause a spiraling peak in the eye of the cyclone that can force the filtered water to move up towards the surface, enabling the manta rays to catch their prey without difficulty. This motion of manta rays in d -dimensional search space can be expressed as follows [7, 22]:

$$x_i^{t+1} = \begin{cases} x_{best,d}^t + r \cdot (x_{best,d}^t - x_{i,d}^t) + \beta \cdot (x_{best,d}^t - x_{i,d}^t) & i = 1 \\ x_{best,d}^t + r \cdot (x_{i-1,d}^t - x_{i,d}^t) + \beta \cdot (x_{best,d}^t - x_{i,d}^t) & i = 2, \dots, N \end{cases} \quad (11)$$

$$\beta = 2e^{r_1} \frac{T_{max} - t + 1}{T_{max}} \cdot \sin(2\pi r_1) \quad (12)$$

where β is a weight coefficient and r_1 is a random number between 0 and 1.

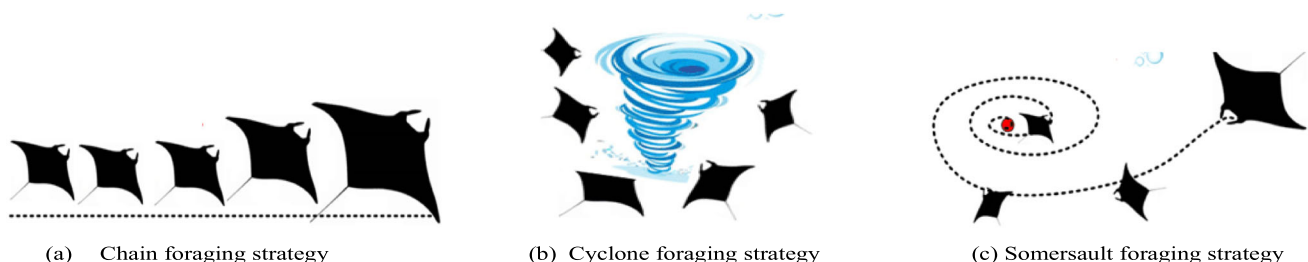


Fig. 6 Three foraging strategies of the manta rays

MRFO balances exploitation and exploration by using the best solution as a reference point for exploitation. To promote exploration, individuals are randomly repositioned far from their current and best positions, as described in [7, 22]:

$$x_{rand,d} = lb_d + r \cdot (ub_d - lb_d) \quad (13)$$

$$x_{i,d}^{t+1} = \begin{cases} x_{rand,d}^t + r \cdot (x_{rand,d}^t - x_{i,d}^t) + \beta \cdot (x_{rand,d}^t - x_{i,d}^t) & i = 1 \\ x_{rand,d}^t + r \cdot (x_{i-1,d}^t - x_{i,d}^t) + \beta \cdot (x_{rand,d}^t - x_{i,d}^t) & i = 2, \dots, N \end{cases} \quad (14)$$

3. Somersault foraging strategy: When a food source is found, the manta rays do a number of backward somersaults and circle the food to drag it toward their open mouths. The positions of the manta rays are updated around the best position obtained up to now, as follows [7, 22]:

$$x_{i,d}^{t+1} = x_{i,d}^t + s \cdot (r_2 \cdot x_{best,d}^t - r_3 \cdot x_{i,d}^t) \quad (15)$$

where s is the somersault factor, whose value is set to 2 and controls the somersault range of manta rays, and r_2 and r_3 are randomly generated numbers between 0 and 1. Figure 7 illustrates the pseudo-code of the MRFO.

3.2 Enhanced manta ray foraging optimization algorithms

This study proposes four enhanced variants of the MRFO algorithm: opposition-based MRFO (OMRFO), quasi-oppositional MRFO (QMRFO), opposition-based generation jumping MRFO (JOMRFO), and quasi-oppositional generation jumping MRFO (JQMRFO). These enhancements incorporate three strategies: opposition-based learning (OBL), quasi-oppositional learning (QOBL), and jumping rate (J_r). These strategies aim to refine the

```

Initialize the algorithm parameters: max. number of iterations ( $T_{max}$ ) and population size ( $N$ )
Define the problem dimension ( $d$ ) and upper and lower bounds of each problem variable ( $lb_j, ub_j$ )
Initialize randomly the positions of manta rays ( $x$ ) within the predefined bounds,  $t=1$ 
Compute the fitness value of each manta ray using objective function
Select the fittest vector based on the fitness value
While the stopping criterion is not met Do
    For  $i=1:N$ 
        If  $r < 0.5$  Apply cyclone foraging strategy
            If  $t/T_{max} < \text{rand}$ 
                Apply cyclone foraging strategy utilizing Eq. (14)
            Else
                Apply cyclone foraging strategy utilizing Eq. (11)
            End If
        Else
            Apply chain foraging strategy utilizing Eq. (9)
        End If
        Check the feasibility of manta rays and regenerate the violated solutions
        Compute the fitness values of the manta rays and extract the best solution
        Apply the somersault strategy utilizing Eq. (15)
        Check the feasibility of manta rays and regenerate the violated solutions
        Compute the fitness values of the manta rays and extract the best solution
        Update the manta ray positions and update the best position
    End For
End While
Return the best solution achieved thus far
    
```

Fig. 7 Pseudo-code of conventional MRFO

MRFO's population initialization and solution generation steps, leading to enhanced exploration and exploitation capabilities.

3.2.1 Opposition-based learning (OBL)

Opposition-based learning (OBL) [7, 30] seeks to expedite the convergence rate of heuristic optimization algorithms by exploring the search space using opposite solutions. The core principles of OBL revolve around opposite numbers and points. The opposite of a real number y within an interval $[m, n]$ is calculated as:

$$y^o = m + n - y \quad (16)$$

In a d -dimension space, the opposite point y_i^o for each dimension i is computed as:

$$y_i^o = m_i + n_i - y_i; \quad i = 1, 2, \dots, d \quad (17)$$

Incorporating OBL into the MRFO algorithm (OMRFO) enhances exploration by expanding the search space and increasing the likelihood of finding superior solutions. This is achieved by comparing the fitness of each solution with its opposite counterpart, retaining the fitter solution to improve subsequent generations. The pseudo-code of the OBL is shown in Fig. 8.

3.2.2 Quasi-oppositional learning (QOBL)

Quasi-oppositional learning (QOBL) [31, 32] extends OBL by generating quasi-opposite solutions strategically positioned between the centre and the opposite of the solution. For a real number y in the interval $[m, n]$, the quasi-opposite number y^{qo} is determined as:

$$y^{qo} = rand\left(\frac{m+n}{2}, m+n-y\right) \quad (18)$$

In a d -dimensional space, the quasi-opposite point y_i^{qo} for each dimension i is calculated as:

$$y_i^{qo} = rand\left(\frac{m_i+n_i}{2}, m_i+n_i-y_i\right); \quad i = 1, 2, \dots, d \quad (19)$$

```

For  $i=1:N$  %population size
  For  $j=1:d$  %No. of variables in the problem
     $X_{i,j}^o = lb_j + ub_j - X_{i,j}$  % $lb_j, ub_j$  are the lower and upper bounds of the variable  $j$ ;
     $X_{i,j}$  is the population, and  $X_{i,j}^o$  is opposite-based population
  End
End
For  $z \leq N$  do
  Evaluate  $X_z$  utilizing desired objective function
  Evaluate  $X_z^o$  utilizing desired objective function
  If  $\text{Fit}(X_z^o) < \text{Fit}(X_z)$  % Fit is fitness value
     $X_z = X_z^o$ 
  End If
End for

```

Fig. 8 Pseudo-code of OBL

```

For  $i=1:N$  %population size
    For  $j=1:d$  %No. of variables in the problem
         $X_{i,j}^o = lb_j + ub_j - X_{i,j}$  % $lb_j, ub_j$  are the lower and upper bounds of the variable  $j$ ;
         $X_{i,j}$  is current population, and  $X_{i,j}^o$  is opposite-based population
         $M_{i,j} = \frac{lb_j + ub_j}{2}$ 
        if  $X_{i,j}^o < M_{i,j}$ 
             $X_{i,j}^q = X_{i,j}^o + (M_{i,j} - X_{i,j}^o) \times rand$  % $X_{i,j}^q$  is quasi-oppositional-based population
        else
             $X_{i,j}^q = M_{i,j} + (X_{i,j}^o - M_{i,j}) \times rand$ 
        End
    End
End

```

Fig. 9 Pseudo-code of QOBL

QOBL steers the search process toward more promising regions, improving convergence rates. The pseudo-code of the QOBL is illustrated in Fig. 9.

3.2.3 Generation jumping

To further enhance the exploration, the generation jumping mechanism integrates OBL and QOBL with a probabilistic jumping rate (J_r) parameter [32, 33]. This parameter determines the frequency of generating the OBL/QOBL-based populations. Fittest solutions are then selected from the combined current and OBL/QOBL populations. Notably, quasi-oppositional generation jumping employs a lower J_r compared to opposition-based generation jumping ($J_{r-QOBL} = \frac{1}{6}J_{r-OBL}$) to prevent premature convergence and maintain population diversity [32]. In this study, the parameters are set as $J_{r-OBL} = 0.3$, and $J_{r-QOBL} = 0.05$ [32]. Figure 10 illustrates the pseudo-code for generating the OBL/QOBL generation jumping populations.

```

% opposition-based generation jumping
If  $rand(0,1) < J_r$ 
    For  $i=1:N$  %population size
        For  $j=1:d$  %No. of variables in the problem
             $X_{i,j}^o = lb_j + ub_j - X_{i,j}$  %  $X_{i,j}^o$  is opposite-based (OBL) population
        End % End of oppositional-based generation jumping

%quasi-oppositional generation jumping
If  $rand(0,1) < J_r$ 
    For  $i=1:N$  %population size
        For  $j=1:d$  %No. of variables in the problem
             $X_{i,j}^o = lb_j + ub_j - X_{i,j}$ 
             $M_{i,j} = \frac{lb_j + ub_j}{2}$ 
            if  $X_{i,j}^o < M_{i,j}$ 
                 $X_{i,j}^q = X_{i,j}^o + (M_{i,j} - X_{i,j}^o) \times rand$  %  $X_{i,j}^q$  is quasi-oppositional-based (QOBL) population
            else
                 $X_{i,j}^q = M_{i,j} + (X_{i,j}^o - M_{i,j}) \times rand$ 
            End
        End % End of quasi-oppositional generation jumping
    End

```

Fig. 10 Pseudo-code of OBL/QOBL generation jumping populations

3.3 Addressing the local optima challenge in MRFO

The standard MRFO algorithm is often challenged by its susceptibility to local optima, particularly in complex, multimodal optimization problems. This limitation arises from an inherent imbalance between exploration and exploitation phases. Enhanced variants (OMRFO, QMRFO, JOMRFO, and JQMRFO) address this issue by incorporating specific mechanisms to improve search diversity, enhance exploration, and avoid premature convergence:

1. Opposition-based learning strategies (OMRFO, QMRFO): These variants generate opposite or quasi-opposite solutions for each candidate, enabling exploration of less-frequented regions of the search space. By systematically evaluating opposite or quasi-opposite values, these variants preserve population diversity and reduce stagnation around local optima.
2. Generation jumping (JQMRFO and JOMRFO): This mechanism facilitates a more aggressive exploration of the search space by allowing the algorithm to “jump” to unexplored regions based on probabilistic criteria, reducing the risk of stagnation in local optima and encourages convergence towards global optima.

By effectively balancing exploration and exploitation, these enhancements enable the MRFO algorithm to overcome stagnation and achieve more robust performance.

3.4 Computational complexity analysis

The computational complexity of MRFO and its enhanced variants depends on population size (N), search space dimensionality (d), and the maximum number of iterations (T_{max}) [22]. The computational complexity of the MRFO algorithm can be decomposed into three primary components: initialization, fitness evaluation, and solution updating. During the initialization phase, generating a population of N solutions within a d -dimensional search space incurs a time complexity of $O(Nd)$. Subsequently, evaluating the fitness of this population requires $O(N)$ time. The process of updating solutions in each iteration—through foraging phases such as somersault foraging combined with chain or cyclone foraging—exhibits a time complexity of $O(Nd)$. Consequently, over T_{max} iterations, the overall time complexity for the MRFO algorithm can be expressed as $O(T_{max}(Nd + Nd)) = O(T_{max}Nd)$ [22, 25].

A similar analysis applies to the enhanced variants OMRFO and QMRFO, which integrate OBL and QOBL, respectively. The time complexity associated with OBL/QOBL is $O(Nd)$ per iteration [26]. Therefore, the total computational complexity for these algorithms can be expressed as $O(T_{max}Nd)$.

In addition, JOMRFO and JQMRFO utilize a jumping rate mechanism that adds a negligible complexity of $O(N)$ per iteration due to probabilistic checks [33], leaving the total asymptotic complexity unchanged; thus: $O(JOMRFO) = O(JQMRFO) = O(T_{max}Nd)$.

3.5 Memory requirements

In standard MRFO, the primary memory requirements include storage for a population of N candidate solutions each with d dimensions ($O(Nd)$), fitness values for the population ($O(N)$), the best solution ($O(d)$), and temporary variables for computations during solution updates ($O(d)$). Therefore, the total space complexity is $O(Nd)$.

The integration of opposition-based learning techniques introduces opposite solutions for the entire population, effectively doubling the population size during iterations. This results in an additional $O(Nd)$ storage requirement for the opposite solutions. Consequently, the total asymptotic space complexity of OMRFO and QMRFO is $O(2Nd) = O(Nd)$.

The inclusion of the jumping rate parameter introduces negligible overhead. Therefore, the overall space complexity of JOMRFO and JQMRFO remains $O(Nd)$.

4 Implementation of the proposed algorithms for economic dispatch problems

The output power from generation unit functions as control variables in ED problems. The proposed algorithms utilize these variables to model the population and assess each solution using the objective function. To address the equality constraint, a slack generator is designated as a dependent variable. The execution of these algorithms for ED problems follows these steps:

- Step 1:** Define the algorithm parameters, including the maximum number of iterations (T_{max}) and population size (N). Also, input the test system data.
- Step 2:** Initialize the iteration counter, t , to 1. Randomly initialize the active power output of all generation units, excluding the slack unit (last unit), within their predefined range using Eq. (20). The slack unit is initialized using Eq. (21) to ensure that the equality constraint is satisfied. Subsequently, the active power of the slack unit (P_s) is checked to ensure it falls within its predefined limits. If P_s violates the inequality constraint, the corresponding solution is penalized. The initial population (P_{ini}) is then created depending on the initial values of active power, as illustrated in Eq. (22).

$$P_j = P_j^{min} + rand \times (P_j^{max} - P_j^{min}) \quad (20)$$

$$P_s = P_D + P_{loss} - \sum_{j=1}^{Ng-1} P_j \quad (21)$$

$$P_{ini} = \begin{bmatrix} P_{1,1}, & P_{1,2}, & \dots & P_{1,j}, & \dots & P_{1,Ng} \\ P_{i,1}, & P_{i,2}, & \dots & P_{i,j}, & \dots & P_{i,Ng} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ P_{N,1}, & P_{N,2}, & \dots & P_{N,j}, & \dots & P_{N,Ng} \end{bmatrix} \quad i = 1, \dots, N; \quad j = 1 : Ng \quad (22)$$

where $P_{i,j}$ is the active output power from generator j in solution vector i .

- Step 3:** Generate the OBL/QOBL population, as illustrated in Figs. 8 and 9.
- Step 4:** Evaluate the fitness of solutions in the current and OBL/QOBL populations using Eq. (23). Select the fittest vectors from these populations based on their fitness values.

$$F(P_{gn}) = \sum_{n=1}^{Ng} F_n(P_{gn}) + \lambda_p (P_s - P_s^{lim})^2 \quad (23)$$

where λ_p is a penalty factor for the slack power and P_s^{lim} is the limit of power generated from the slack unit, which can be determined as follows:

$$P_s^{lim} = \begin{cases} P_s^{max} & \text{if } P_s > P_s^{max} \\ P_s^{min} & \text{if } P_s < P_s^{min} \\ P_s & \text{if } P_s^{min} \leq P_s \leq P_s^{max} \end{cases} \quad (24)$$

where P_s^{max} and P_s^{min} are the slack unit's maximum and minimum output power, respectively.

- Step 5:** Enter the algorithm's iterative loop. For each manta ray (solution), generate a random number, r . If r is less than 0.5, apply the cyclone foraging strategy (Eqs. 14 or 11) to update the manta ray's position. Otherwise, apply the chain foraging strategy (Eq. 9).

- Step 6:** Verify system constraints by assessing the feasibility of each generator's output power using inequality constraints (7), excluding the slack generator. If any violation is detected, the violated solution is regenerated. Additionally, determine and verify the feasibility of the slack unit's output power, following the procedure outlined in step 2. Any violation of system constraints results in a non-feasible solution. In such cases, the violated solution is penalized.
- Step 7:** Calculate the fitness value for each solution and identify the solution with the highest fitness.
- Step 8:** Implement the somersault strategy, utilizing Eq. (15) to update the solution.
- Step 9:** Check the feasibility of solutions and regenerate the violated solutions.
- Step 10:** After generating the updated manta ray population, generate the OBL/QOBL population using jumping rate, and calculate its fitness. Also, calculate the fitness of the updated manta ray population.
- Step 11:** Identify the best solution from the combined set of updated and OBL/QOBL populations by comparing their fitness values and choosing the one with the highest fitness.
- Step 12:** Update the positions of the manta rays, which represent output power, and also update the best solution, which represents the lowest fuel cost achieved so far.
- Step 13:** If the stop criterion is not satisfied ($t < T_{max}$), proceed to step 5. Otherwise, return the best solution found during the optimization process.

5 Simulation results and discussions

This section presents a comprehensive evaluation of the MRFO and its four proposed enhanced variants, including OMRFO, QMRFO, JOMRFO, and JQMRFO. The performance assessment was conducted across five benchmark test systems representing varying complexities: a 10-unit system, a 15-unit system, a 40-unit system, a 140-unit system, and a 160-unit system. These systems were chosen for their diverse characteristics, including varying load demands, power losses, VPL effects, POZ, and MFO. To ensure a rigorous comparison, five test cases were analysed based on these system characteristics. The proposed algorithms were compared against those found in published literature. To assess the algorithms' stability and robustness, each test case was run multiple times: 20 individual runs for test cases 1, 2, and 3, and 50 runs for test cases 4 and 5. The best, mean, and worst fuel cost values, along with the standard deviation (StD), were recorded for each algorithm and test case. The best results of each case are marked in bold, indicating the best statistical results and the minimum cost, highlighting the most optimal solutions obtained. To ensure a fair comparison, the control parameters (N and T_{max}) for all algorithms, including MRFO and its enhanced variants, were optimized using the MRFO algorithm itself. Details of this tuning process are provided in Section 5.4 (Tuning Parameters of MRFO Algorithm). All simulations were implemented using MATLAB (R2020a) on a personal computer with an 11th Generation Intel(R) Core (TM) i7-1165G7 CPU, 16 GB RAM, and the Windows 11 operating system.

5.1 Benchmark test systems

This study employs five benchmark test systems to evaluate the performance of the proposed enhanced MRFO algorithms. These systems represent a range of complexities and data sources, including synthetic and real-world models, allowing for comprehensive testing of the algorithms' effectiveness.

1. Test system 1: standard 10-unit test system

This widely used synthetic system represents a typical power generation system. It includes 10 generating units with constraints such as VPL and transmission power losses. The network losses are calculated by the B -matrix loss formula [13]. The data for the test system is sourced from [13]. The total load demand is 2000 MW. This system is chosen for its simplicity and the availability of comparative results from previous studies, allowing for a baseline evaluation of the algorithms' performance.

2. Test system 2: Standard 15-unit test system

This synthetic system comprises 15 generating units with a load demand of 2630 MW. Four units (units 2, 5, 6, and 12) feature POZs, while the remaining units have convex cost functions. Transmission power losses are also taken into account. The data for the test system is sourced from [15].

3. Test system 3: Standard 40-unit test system

This system is based on a real-world scenario from the Taiwan Power Company (Taipower) and consists of 40 generating units, a mix of coal-fuelled and oil-fuelled units. The load demand is 10,500 MW. For a more realistic representation, VPL effects are considered in the fuel cost function. The fuel cost coefficients of the generating units are taken from [15].

4. Test system 4: Standard 140-unit test system

This real-world system represents the large-scale Korean power system. The test system involves 140 generating units with a total load demand of 49,342 MW. The generation units are divided into four types: 40 thermals, 41 gas, 20 nuclear, and 21 oil. Among these units, 6 thermals, 4 gas, and 2 oil feature non-convex fuel cost functions with VPL effects. The data of the test system are adopted from [11].

5. Test system 5: Standard 160-unit test system

This synthetic system is formed by duplicating a basic 10-unit test system with piecewise cost functions, VPL effects, and a load demand of 2700 MW, scaled proportionally to the system size. The system demand is 43,200 MW. Transmission power losses are neglected. The input data is sourced from [34].

5.2 Case studies

1. Test Case 1

Test case 1 utilizes a 10-unit system, considering VPL effects and transmission power losses. Table 2 presents the best solutions obtained by MRFO and its four enhanced variants, compared against recently published algorithms: GJO [5], LWSO [5], SGO [13], MSGO [13], GNDO [14], and Ch-JAYA [35]. Among these, JOMRFO achieves the minimum fuel cost of 111,466.57126 \$/h, outperforming all other algorithms. Table 2 also details the algorithms' CPU times. Although MRFO's CPU time is higher than SGO and MSGO, it is lower than GJO, GNDO, Ch-JAYA, and its enhanced algorithms. Moreover, JOMRFO demonstrates a slightly lower CPU time than OMRFO, QMRFO, and JQMRFO in the range of milliseconds. Finally, Table 2 illustrates the enhanced algorithms' capability to handle system constraints and reach optimal solutions effectively.

Statistical results for 20 independent runs, summarized in Table 3, underscore the robustness of the enhanced algorithms. All four variants achieve lower costs than the conventional MRFO, highlighting the effectiveness of the proposed enhancements in avoiding local optima. Notably, the JOMRFO algorithm demonstrates a significantly lower cost compared to other algorithms in Table 3, leading to cost savings ranging from 31.06 \$/h, compared to the second-ranked LWSO method, to 425.84 \$/h, compared to the fifteenth-ranked QOPO method. These hourly savings translate to significant annual cost reductions between 272,075 \$ and 3,730,344 \$, emphasizing the economic benefits of JOMRFO. Moreover, the JOMRFO robustness in fuel cost optimization is highlighted by its consistently superior performance; its worst, mean, and best fuel costs surpass the minimum costs obtained by the other proposed and reported algorithms, indicating its reliable ability to minimize fuel costs.

Figure 11.a illustrates the convergence behaviour of MRFO and its enhanced variants for this test system. While all algorithms converge within 200 iterations, the enhanced variants achieve lower total costs and faster convergence compared to the standard MRFO. Notably, MRFO exhibits a more gradual convergence and stabilizes at a higher total cost, suggesting that it may become trapped in local optima in certain regions of the search space. In contrast, the enhanced variants, particularly JOMRFO, demonstrate improved capabilities to

Table 2 Best solutions for test case 1 obtained by different optimization methods

Unit	MRFO	OMRFO	QMRFO	JQMRFO	JOMRFO	GJO [5]	LWSO [5]	SGO [13]	MSGO [13]	GNDO [14]	Ch-JAYA [35]
1	54.8111	30.0676	55.0000	55.0000	55.0000	55.0000	55.0000	55.0000	55.0000	52.33	55.0000
2	79.5936	80.0000	80.0000	79.5266	29.9219	80.0000	80.0000	80.0000	80.0000	71.22	80.0000
3	113.3767	85.7577	120.0000	96.6659	120.0000	107.5513	106.9388	106.9542	106.93398	97.79	106.9381
4	102.6163	120.9145	101.9266	117.0798	130.0000	100.1761	100.5743	100.5724	100.5786	74.79	100.5886
5	82.2902	95.9058	80.7799	107.4870	90.0126	81.88875	81.5070	81.4829	81.5030	58.40	81.49590
6	80.1371	155.6956	76.6439	70.0000	109.1000	82.40525	83.0188	83.0293	83.0232	220.32	83.01620
7	299.4683	249.9142	300.0000	298.2247	300.0000	300.0000	300.0000	299.9999	300.0000	293.93	300.0000
8	339.8106	340.0000	340.0000	337.9880	340.0000	340.0000	340.0000	340.0000	340.0000	297.49	340.0000
9	469.8360	470.0000	470.0000	467.2186	470.0000	470.0000	470.0000	470.0000	470.0000	461.97	470.0000
10	469.9995	463.1708	467.4772	462.2121	447.2398	470.0000	470.0000	470.0000	470.0000	458.24	470.0000
V* (MW)	0.0	0.0	0.0	0.0	0.0	0.021904	1.47e-06	-9.9e-6	-9.8e-6	-	1.466e-12
Loss (MW)	91.9392	91.4259	91.8276	91.4027	91.2744	87.0433	87.0388	87.0389	87.0388	85.4900	87.0388
Min Cost (\$/h)	111,823,2656	111,586,3296	111,587,268	111,591,8812	111,466,5713	111,498.46	111,497.63	111,497,6302	11,497,6301	113,921.95	111,497,6312
CPU Time (s.)	3.287	4.607	4.644	4.701	4.567	-	-	0.17	0.18	9.48	-

*Violation of power balance constraint

Table 3 Statistical results for test case 1 over 20-run

Algorithm	Best cost	Mean cost	Worst cost	Std	Cost savings (\$/h)	Rank
MRFO	111,823.2656	111,846.13148	111,875.4325	13.4446	356.6943	14
OMRFO	111,586.3296	111,683.8244	111,794.9940	82.21995	119.7583	11
QMRFO	111,587.2689	111,656.9723	111,798.0904	68.747	120.6977	12
JQMRFO	111,591.8812	111,662.8153	111,787.469	79.7951	125.31002	13
JOMRFO	111,466.5712	111,476.7955	111,492.3204	10.17460	0.0	1
LWSO [5]	111,497.63	111,497.63	111,497.63	0.000162	31.05874	2
WSO [5]	111,497.8	111,497.8	111,497.9	0.028971	31.22874	6
GJO [5]	111,498.5	111,499.8	111,501.6	0.978837	31.92874	8
NGO [5]	111,497.7	111,498	111,498.3	0.152862	31.12874	5
FOX [5]	111,498.2	111,618.3	112,266.3	223.1766	31.62874	7
MSGO [13]	111,497.6301	111,497.6302	111,497.6304	0.0000625	31.058839	3
SGO [13]	111,497.6302	111,497.7362	111,502.7703	0.0727	31.05894	4
HHO [36]	111,503.9	111,566.9	111,664.6	35.06826	37.328739	9
JS [36]	111,505.9	111,508.9	111,513.3	1.578434	39.328739	10
QOPO [10]	111,892.4096	—	—	—	425.83834	15

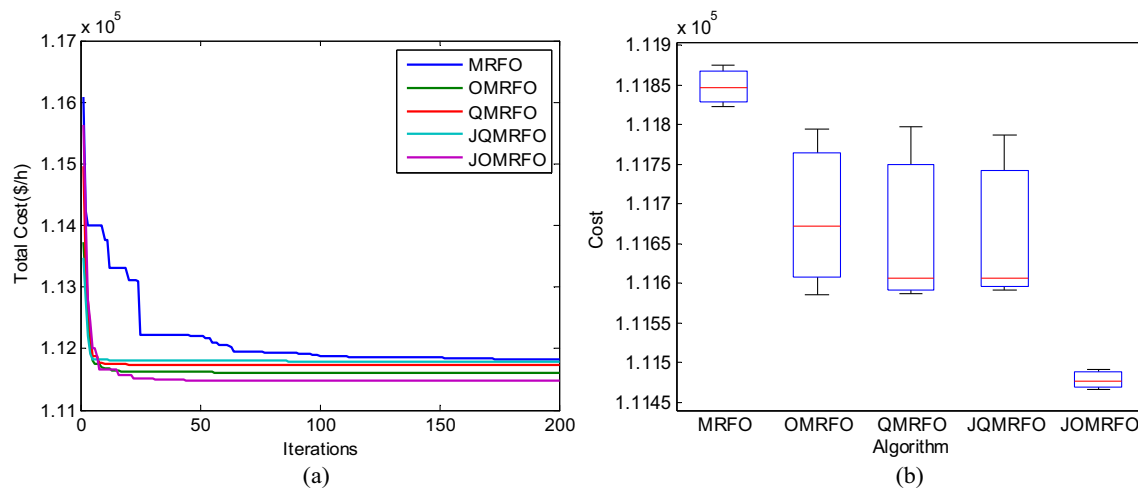


Fig. 11 Convergence characteristics and boxplot of test case 1

escape local optima, achieving better solutions within fewer iterations. These results highlight the importance of the modifications in the enhanced variants, which improve both convergence speed and solution quality.

Figure 11b uses boxplots to visualize the distribution of fuel cost results for the proposed algorithms. The median fuel cost is represented by a line within each box, while whiskers extend to show the minimum and maximum values. A wider and taller box indicates greater variability in fuel costs, while a narrower box suggests more consistent results. Analysis of the boxplots demonstrates that the JOMRFO algorithm has the smallest box length with the lowest median, indicating higher stability. Moreover, MRFO also shows a relatively small box length. The OMRFO and QMRFO have slightly larger boxes, suggesting less stability. JQMRFO falls in between these categories.

2. Test Case 2

Test case 2 uses a 15-unit system, considering transmission power losses and prohibited operating zones (POZs), detailed in Table 4. Table 5 presents the best solutions obtained by MRFO and its four enhanced variants, compared against recent algorithms: Jaya-SM [19], HGS [15], GBO [6], RUN [6], GWO [6], SCA-βHC [17], and EO-SCA [37]. JQMRFO achieves the lowest fuel cost of 32,575.54273 \$/h, with JOMRFO, OMRFO, QMRFO, and MRFO all achieving nearly identical results, outperforming all other listed algorithms. The proposed algorithms also demonstrate superior performance in handling system constraints to achieve optimal solutions. Regarding the execution time, MRFO achieves the lowest execution time, second only to JAYA-SM. Furthermore, the four enhanced algorithms outperform other reported algorithms (GWO, RUN, GBO, HGS, SCA-βHC, and EO-SCA) in terms of execution time.

The statistical results in Table 6 further validate the robustness of the proposed enhancements. The results show that MRFO and its four enhanced variants all achieve lower costs compared to other reported algorithms. JQMRFO achieves the minimum fuel cost, ranking first and outperforming other algorithms, highlighting its superior balance of exploration and exploitation. These results demonstrate significant cost savings, ranging from 0.00002 \$/h for JOMRFO, ranked second, to 342.32730 \$/h for TS, ranked 34th, resulting in substantial annual cost reductions up to 2,998,787.15 \$.

Figure 12a illustrates the convergence characteristics of the algorithms. While MRFO exhibits slow convergence and stagnates, the enhanced algorithms—especially JQMRFO and JOMRFO—demonstrate a more refined and continuous decrease in fuel costs, signifying effective local optima avoidance. Figure 12b presents boxplots for the proposed algorithms. As shown, the five algorithms have very similar median fuel costs and a very narrow

Table 4 Prohibited operating zones for 15-unit test system

Unit	Zone 1	Zone 2	Zone 3
2	[185 255]	[305 335]	[420 450]
5	[180 200]	[305 335]	[390 420]
6	[230 255]	[365 395]	[430 455]
12	[30 55]	[65 75]	–

range of fuel costs, demonstrating that all five algorithms are very stable and effective for optimizing fuel costs for this test case.

3. Test case 3

Test Case 3 examines the medium-scale realistic 40-unit test system with VPL effects. Table 7 presents the best solutions obtained by MRFO and its four enhanced variants, compared against Jaya [19], ESCSDO10 [15], HGS [15], HSSA [44], QOPO [10], PPSO [48], ESNS [6], and GBO [6]. The results reveal that the proposed JOMRFO achieves the best fuel cost of 119,265.5052 \$/h, outperforming all other proposed and reported methods in Table 7. The CPU time taken by the proposed algorithms is also listed in Table 7, showcasing JOMRFO's notable efficiency compared to others. Additionally, the capability of the proposed algorithms in handling generation and demand equality constraints is verified via Table 7. Table 8 presents a comparison of statistical results over 20 runs. The statistical analysis in Table 8 consistently shows that JOMRFO achieves the best fuel cost and the least worst fuel cost compared to other methods, indicating its effectiveness in producing superior results. JOMRFO's adaptive jumping mechanism and oppositional strategies enable effective exploration of search space, allowing it to avoid local optima and deliver substantial cost savings. Compared to other methods, savings range from 182,4785 \$/h for the JQMRFO, ranked in 2nd position, to 6237.6 \$/h for the PPSO, ranked in the 40th position, resulting in significant annual cost reductions of 1,598,511.66 \$ to 54,641,376 \$, demonstrating the economic advantage of JOMRFO over other optimization methods.

Figure 13a highlights JOMRFO's superior convergence behaviour, consistently achieving lower fuel costs than other methods by avoiding stagnation and refining its solutions continuously. Boxplots in Fig. 13b confirm JOMRFO's stability, with minimal variability in fuel costs, while QMRFO exhibits greater variation. These results demonstrate JOMRFO's reliability and effectiveness in optimizing complex medium-scale systems.

4. Test Case 4

In this case, the proposed algorithms are applied to the realistic 140-unit test system with VPL effects. Table 9 summarizes the comparison results over 50 individual runs. The JOMRFO algorithm ranked first by achieving a minimum fuel cost of 1,549,308.7457 \$/h, significantly outperforming conventional MRFO (1,559,900.00 \$/h), OMRFO (1,555,700.00 \$/h), QMRFO (1,550,885.57 \$/h), JQMRFO (1,550,061.4190 \$/h), and all other listed algorithms. This performance leads to substantial cost savings ranging from 752.67329 \$/h for the JQMRFO, ranked in 2nd position, to 109,075.50429 \$/h for the SSA, the algorithm with the worst performance. This results in significant annual cost reductions from 6,593,418.0204 \$ to 955,501,417.5804 \$. The statistical results confirm JOMRFO's superiority, achieving the best mean, worst-case, and minimum fuel costs, with a competitive standard deviation ranking fourth after haDEPSO [59], aPSO [59], and MTVPSO [60]. Additionally, JOMRFO demonstrates a competitive average CPU time of 49.96 s, which is faster than most algorithms in Table 9, although slower than DHS [59], aPSO [59], and haDEPSO [59]. The best generation schedule achieved by the MRFO and JOMRFO algorithms for this test case is shown in Table 10.

Figure 14a illustrates the convergence characteristics of the proposed algorithms. Although the proposed algorithms show similar convergence patterns, the JOMRFO maintains consistent progress and converges to a

Table 5 Best solutions for test case 2 obtained by different optimization methods

Unit	MRFO	OMRFO	QMRFO	JQMRFO	JOMRFO	Jaya-SM [19]	HGS [15]	GBO [6]	RUN [6]	GWO [6]	SCA- BHC [17]	EO-SCA [37]
1	454.0953	411.5149	284.4460	356.8639	454.8564	455	455	454.9998	455	452.9496	454.9992	455
2	455.0000	418.5980	455.0000	454.8938	450.0000	380.0	380	379.9995	379.9748	378.1478	379.9906	380
3	130.0000	130.0000	130.0000	112.7281	130.0000	130	130	129.9998	130	129.8193	129.9949	129.99
4	130.0000	130.0000	130.0000	95.3451	130.0000	130	130	130	130	130	129.9893	130
5	260.0000	382.7877	470.0000	368.3026	150.0000	170	169.9999	152.4773	169.9998	164.2171	151.3573	170
6	459.6268	349.9746	460.0000	411.4472	460.0000	460	460	459.9973	459.9864	458.7296	455.7416	460
7	465.0000	465.0000	465.0000	465.0000	465.0000	430	430	430	429.9992	428.9978	429.8619	430
8	60.0000	60.0000	60.0000	60.0000	60.0000	60.9089	60	60.07531	60.02533	67.02907	126.1459	67.91
9	25.0000	80.2539	25.0000	76.3300	25.0000	69.9168	69.55549	88.13783	101.355	83.29008	68.6827	62.38
10	20.0000	62.8445	25.3508	41.3384	107.6975	160	160	159.2346	127.3228	134.238	110.4942	160
11	80.0000	41.0707	43.6140	23.2433	80.0000	80	80	80	79.99605	78.06923	72.5522	80
12	63.0257	55.0000	62.7860	55.4291	64.8039	80	80	79.99992	79.78253	73.3213	79.7710	79.99
13	25.0000	25.0000	25.0000	63.5235	25.0000	25	25	25.01132	25.33732	25.1814	31.8292	25
14	15.0402	34.5874	15.0000	24.7641	35.0258	15	15	15.03286	15.49778	26.74054	17.1081	15
15	15.0000	15.0000	15.0000	52.2928	17.7683	15	15	15.37101	15.02823	28.58303	21.9895	15
Total power generated(MW)	2656.7880	2661.631	2666.197	2661.50187	2655.1518	2660.8257	2659.55539	-	-	-	2660.51	2660.27
V (MW)	0.0	0.0	0.0	0.0	0.0	0.0031	0.0	9.72E-11	0.000188	0.003361	0.0	0.0
Loss (MW)	26.7880	31.631	36.197	31.5019	25.1518	30.8288	29.55539	30.3366	29.3055	29.3104	30.5077	30.295
Min Cost (\$/h)	32,575.54278	32,575.54276	32,575.54276	32,575.54273	32,575.54275	32,706.9830	32,692.64264	32,717.26	32,700.58	32,740.74	32,761.56	32,700.51
CPU Time (s.)	6.217	8.623	8.649	8.723	8.457	5.14	-	99.8369	119.648	48.5059	-	-

Table 6 Statistical results for test case 2 over 20-run

Algorithm	Best cost	Mean cost	Worst cost	StD	Cost savings (\$/h)	Rank	Algorithm	Best cost	Mean cost	Worst cost	StD	Cost savings (\$/h)	Rank
MRFO	32,575.54278	32,575.5801	32,575.9131	0.0943	0.00005	4	RUN [6]	32,700.58	32,730.47	32,745.06	12.96173	125.0373	13
OMRFO	32,575.54276	32,575.5461	32,575.6355	0.0166	0.00003	3	ESNS [6]	32,693.08	32,696.05	32,698.71	1.639649	117.5373	8
QMRFO	32,575.54276	32,575.5954	32,575.9133	0.1257	0.00003	3	SNS [6]	32,724.42	32,726.41	32,728.27	1.381541	148.8773	29
JQMRFO	32,575.54273	32,575.5584	32,575.9131	0.0679	0.0	1	GWO [6]	32,740.74	32,759.39	32,807.62	13.28598	165.1973	31
JOMRFO	32,575.54275	32,575.5585	32,575.9131	0.0679	0.00002	2	EO-SCA [37]	32,700.51	32,702.74	32,701.05	-	124.9673	12
EO [38]	32,701.18	32,701.31	32,701.51	-	125.6373	14	FA [39]	32,704.5	32,856.1	33,175	-	128.9573	17
PSOSIF[40]	32,706.88	32,707.79	32,709.92	3.04	131.3373	22	EPSO [41]	32,704.83	32,725.37	32,762.01	-	129.2973	18
IA_EDP [42]	32,698.20	32,750.22	32,823.78	29.2989	122.6573	11	GABC[43]	32,706.66	32,706.69	32,706.81	0.035838	131.1173	21
EMA [44]	32,704.45	32,704.45	32,704.45	-	128.9073	16	TLBO[45]	32,697.22	32,697.22	32,697.22	0	121.6773	10
MPSO-GA [46]	32,702.0	32,701.31	32,755.19	-	126.4573	15	GAAP [34]	32,732.95	32,735.06	32,756.01	-	157.4073	30
Jaya [19]	32,712.6458	32,743.4613	32,822.9993	47.0256	137.1031	26	TS [47]	32,917.87	33,066.76	33,245.54	66.82	342.3273	34
Jaya-M [19]	32,707.0312	32,714.4386	32,743.6808	12.0972	131.4885	24	DSPSO-TS [47]	32,715.06	32,724.63	32,730.39	8.4	139.5173	27
Jaya-SM [19]	32,706.983	32,709.0463	32,728.2292	8.7817	131.4403	23	CSO [18]	32,709.36	32,712.49	32,722.55	4.56	133.8173	25
Jaya-SML [19]	32,706.3578	32,706.6764	32,707.2925	2.3244	130.8151	19	CCSO[18]	32,706.64	32,706.64	32,706.64	0.0007	131.0973	20
ESCSO10 [15]	32,692.4	32,730.71	32,740.36	19.64	116.8573	5	HHO [16]	32,863.05	-	33,153.54	153.2478	287.5073	33
HGS [15]	32,692.64	32,697.3	32,725.4	8.948	117.0973	7	HHO- $\alpha\beta$ HCI[16]	32,694.73	-	32,698.74	2.2276	119.1873	9
SDO [15]	32,692.41	32,732.64	32,740.32	17.4465	116.8673	6	SCA- β HC [17]	32,761.56	-	32,799.42	15.7	186.0173	32
GBO [6]	32,717.26	32,731.07	32,760.49	14.24217	141.7173	28							

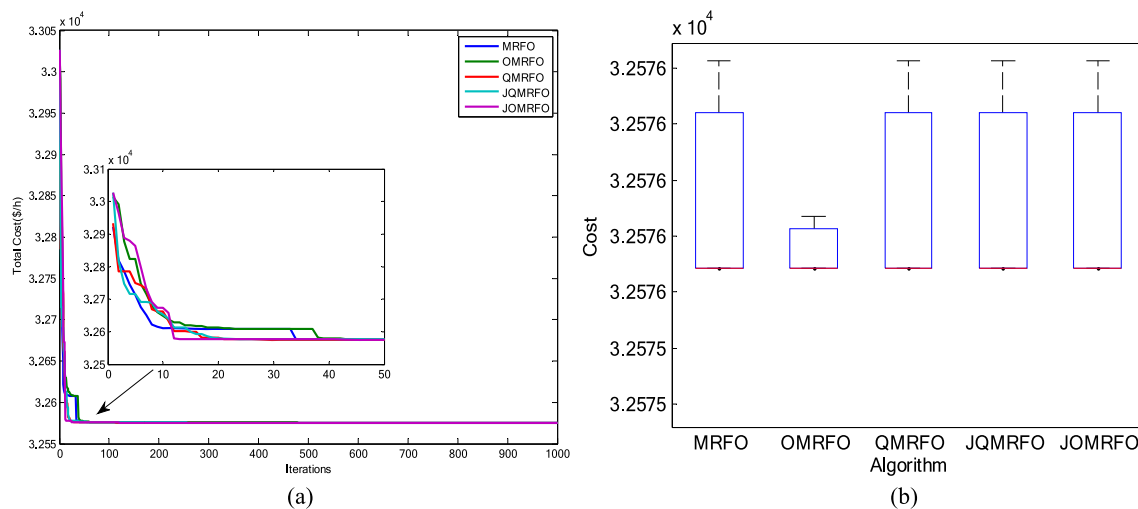


Fig. 12 Convergence characteristics and boxplot of test case 2

lower fuel cost. Figure 14b confirms the JOMRFO's stability, showing the smallest variation among all algorithms. These results establish JOMRFO as a robust and efficient algorithm for large-scale economic dispatch problems.

5. Test Case 5

The 160-unit test system with MFO and VPL effects is utilized in this test case. The best, mean, and worst fuel costs attained by MRFO, OMRFO, QMRFO, JQMRFO, JOMRFO, and other reported algorithms such as OIWO [9], ED-DE [64], CGA-MU [64], IGA-MU [64], ORCCRO [65], BBO [66], and DE/BBO [66] over 50 runs are listed in Table 11. The results clearly prove that JOMRFO is better at achieving the optimal solution for large-scale test systems, outperforming the other methods. The JOMRFO algorithm achieves a minimum fuel cost of 9,981.37701 \$/h, surpassing conventional MRFO (9,981.7620 \$/h), OMRFO (9,981.4866 \$/h), QMRFO (9,981.4761 \$/h), JQMRFO (9,981.4483 \$/h), and the other listed algorithms, leading to substantial cost savings ranging from 0.0317 \$/h for the OMRFO, ranked in 2nd position, to 162.3530 \$/h for the CGA-MU, ranked in the 12th position, subsequently resulting in significant annual cost reductions ranging from 277.692 \$ to 1,422,212.28 \$. The statistical analysis of the 50 runs also indicates that JOMRFO has the best mean, worst-case, minimum fuel costs, and lowest standard deviation of the different algorithms used in this test. Additionally, it demonstrates acceptable average CPU time, further enhancing its efficacy. The most economical fuel type and the best generation schedule achieved by the JOMRFO algorithm are listed in Table 12. Figure 15a illustrates the convergence characteristics of the proposed algorithms for this test system, demonstrating the fast convergence properties of JOMRFO over other proposed algorithms. Additionally, it converges to a lower fuel cost. Figure 15b presents boxplots for the proposed algorithms. As observed, JOMRFO has a tighter distribution and less variability compared to the other algorithms. While OMRFO has a wider spread in the boxplot, suggesting less stability in its performance.

5.3 Tuning parameters of MRFO algorithm

The selection of population size (N) and maximum iterations (T_{max}) is critical for the performance of the MRFO and its enhanced variants. A systematic parameter tuning process was employed to determine optimal parameter settings for each system size (10, 15, 40, 140, and 160 units). This involved initial experiments on the smallest

Table 7 Best solutions for test case 3 obtained by different optimization methods

Unit	MRFO	OMRFO	QMRFO	JQMRFO	JOMRFO	Jaya [19]	ESCSO10 [15]	HGS [15]	HSSA [44]	QOPO [10]	PPSO [48]	ESNS [6]	GBO [6]
1	114.0000	114.0000	114.0000	114.0000	114.0000	114.0000	110.8002	114.0000	113.9994	113.7611	110.7998	111.8716	114
2	114.0000	114.0000	114.0000	114.0000	114.0000	113.0278	110.7814	114.0000	113.9999	113.9886	110.7998	111.7829	114
3	120.0000	120.0000	120.0000	120.0000	120.0000	99.9503	97.3987	60.1394	119.9999	119.9993	97.3999	97.40065	120
4	190.0000	190.0000	190.0000	190.0000	190.0000	179.8226	179.7312	179.7998	189.9998	189.8949	179.7331	179.7338	179.7331
5	97.0000	97.0000	97.0000	97.0000	97.0000	96.2457	87.79114	97.0000	97.0000	97.0000	87.7999	88.87531	96.99999
6	140.0000	140.0000	140.0000	140.0000	140.0000	140	139.9984	140.0000	140	139.9986	140.0000	140	140
7	300.0000	300.0000	300.0000	300.0000	300.0000	299.7289	259.5963	300.0000	280.6695	300.0000	259.5997	259.6019	299.9997
8	300.0000	300.0000	300.0000	300.0000	300.0000	285.6087	284.5963	300.0000	290.4195	299.9992	284.5997	284.6011	290.6229
9	300.0000	300.0000	300.0000	300.0000	300.0000	286.6349	284.5915	300.0000	292.5067	299.997	284.5997	284.6005	284.6131
10	300.0000	300.0000	300.0000	141.3968	131.0846	130.2800	130.0107	130.0000	130.0001	130.0000	130.0000	130.0002	279.5997
11	94.5210	94.0000	96.3459	106.6083	102.7183	94.2775	239.0063	94.00006	168.7998	94.0011	94.0000	168.8001	94.00005
12	95.1181	94.0000	105.4856	105.9214	114.5661	94.0287	166.3011	94.0000	168.7998	94.35996	94.0000	168.7998	168.7998
13	126.6281	125.0000	134.2995	130.3701	131.1466	125.016	214.7574	125.0000	214.7598	125.1963	214.7598	214.7599	214.7598
14	203.8899	229.1567	238.2573	134.8281	256.1881	484.6313	304.5057	394.2847	304.5196	304.5059	394.2794	394.2795	304.5196
15	208.2906	191.6383	130.7539	326.5795	255.1479	304.4157	394.2829	394.2796	304.5196	394.4782	394.2794	394.2794	394.2794
16	219.3462	214.0772	242.1923	300.2723	257.9779	394.2211	394.2769	394.2794	394.2793	394.2599	394.2794	304.5196	125.0005
17	500.0000	500.0000	500.0000	500.0000	500.0000	489.6003	489.276	489.3126	489.2812	489.331	489.2794	489.2794	489.2794
18	500.0000	500.0000	500.0000	500.0000	500.0000	489.8317	489.2794	489.2963	489.2795	489.4125	489.2794	489.2799	489.2794
19	550.0000	550.0000	550.0000	550.0000	550.0000	511.8611	511.2789	511.2809	511.2795	511.2939	511.2794	511.2796	511.2794
20	550.0000	550.0000	550.0000	550.0000	550.0000	511.3126	511.2788	511.3354	511.2795	511.4796	511.2794	511.2796	511.2794
21	550.0000	550.0000	550.0000	550.0000	550.0000	524.0884	523.2804	523.5524	523.2794	525.461	523.2794	523.2797	523.2794
22	550.0000	550.0000	550.0000	550.0000	550.0000	523.8282	523.2774	523.366	523.7119	523.6933	523.2794	523.2795	523.2794
23	550.0000	550.0000	550.0000	550.0000	550.0000	523.6836	523.2791	539.7297	523.2794	526.5896	523.2794	523.2795	523.2794
24	550.0000	550.0000	550.0000	550.0000	550.0000	524.0062	523.2744	523.8539	523.2795	524.3047	523.2794	523.28	523.2794
25	550.0000	550.0000	550.0000	550.0000	550.0000	524.5391	523.2783	523.3009	523.8181	524.7312	523.2794	523.2801	523.2794
26	550.0000	550.0000	550.0000	550.0000	550.0000	526.4563	523.2836	523.3843	523.2794	523.5606	523.2794	523.28	523.2794
27	10.0000	10.0000	10.2499	10.6292	10.0822	10.0789	10.00249	10.0000	13.8386	10.0000	10.0000	10.00022	10
28	10.0047	10.4600	10.2512	10.7834	10.0505	10.2151	10.00074	10.0000	11.7804	10.22341	10.0000	10.0001	10
29	10.5331	10.0000	10.2560	10.7721	10.0785	10.6094	10.00303	10.0000	10.1857	10.08921	10.0000	10.00017	10
30	97.0000	97.0000	97.0000	97.0000	97.0000	96.5283	87.79601	97.0000	94.8277	97.0000	87.7999	89.57694	97
31	190.0000	190.0000	190.0000	190.0000	190.0000	189.8846	189.9963	190.0000	189.9999	190.0000	190.0000	189.9999	189.9987
32	190.0000	190.0000	190.0000	190.0000	190.0000	190	189.9999	190.0000	181.897	189.9998	190.0000	190	190
33	190.0000	190.0000	190.0000	190.0000	190.0000	189.7319	189.9845	190.0000	187.162	190.0000	190.0000	190	190
34	200.0000	200.0000	200.0000	200.0000	200.0000	200	164.8044	172.5026	164.8096	200.0000	164.7998	164.8036	199.9993
35	200.0000	200.0000	200.0000	200.0000	200.0000	170.1334	164.8001	200.0000	180.4562	200.0000	194.3973	164.8223	200
36	200.0000	200.0000	200.0000	200.0000	200.0000	199.9047	164.7969	200.0000	199.9853	199.9999	200.0000	164.8137	200
37	110.0000	110.0000	110.0000	110.0000	110.0000	109.4948	89.09699	110.0000	108.2447	109.9804	110.0000	110	109.9999
38	110.0000	110.0000	110.0000	110.0000	110.0000	109.9924	89.11364	110.0000	90.2908	110.0000	110.0000	109.9999	110

Table 7 (continued)

Unit	MRFO	OMRFO	QMRFO	JQMRFO	JOMRFO	Jaya [19]	ESCSDO10 [15]	HGS [15]	HSSA [44]	QOPO [10]	PPSO [48]	ESNS [6]	GBO [6]
39	110.0000	110.0000	110.0000	110.0000	110.0000	110	89.11416	110.000	89.2026	110.000	110.0000	109.9999	110
40	549.6683	549.6678	549.9084	549.8388	548.9593	512.2693	511.2784	511.3023	511.2793	511.4108	511.2794	511.2794	511.2794
V (MW)	0.0	0.0	0.0	0.0	0.0	0.05	0.0	0.0	0.0	0.0	0.0	3.37E-04	4.22E-09
Min Cost (\$/h)	119,737.428	119,733.27	119,823.2582	119,447.9838	119,265.5052	121,733.5492	121,626.97	121,869.54	121,960.27	121,789.6	121,412.5421	121,415.6	122,103.7
CPU Time (s.)	10.406	14.970	14.904	15.094	14.857	9.89	–	–	–	35.9347	–	61.4472	37.2507

Table 8 Statistical results test case 3 over 20-run

Algorithm	Best cost	Mean cost	Worst cost	Cost savings (\$/h)	Rank	Algorithm	Best cost	Mean cost	Worst cost	Cost savings (\$/h)	Rank
MRFO	119,737.4286	120,568.7351	121,375.907	557.7529	4	GWO [6]	121,951.3	122,254.1	122,965.9	2685.8	34
OMRFO	119,733.2728	120,516.0171	120,838.481	467.7648	3	RUN [6]	121,912.6	122,413.3	123,114.8	2647.1	33
QMRFO	119,823.2581	120,789.2373	122,211.107	484.6866	5	HHO [16]	121,731.95	122,343.17	–	2466.4	29
JQMRFO	119,447.9837	120,534.0385	121,390.754	182.4785	2	HHO-ABHC [16]	121,414.83	121,468.74	–	2149.3	11
JOMRFO	119,265.5052	120,560.5159	121,375.474	0.0	1	EBWO [49]	121,600.9	122,012.6	122,180.9	2335.4	25
IA-EDP [34]	121,436.97	121,648.44	121,492.70	2171.5	17	BWO [49]	122,875.8	123,398.6	123,858.9	3610.3	37
Jaya [19]	121,733.5492	122,279.1504	122,707.1277	2468.0	30	ISMA [36]	121,546.89	121,702.82	121,859.73	2281.4	24
Jaya-M [19]	121,516.9603	121,814.4651	122,269.0088	2251.5	23	SMA [36]	121,621.68	121,770.54	121,994.65	2356.2	26
Jaya-SM [19]	121,485.0974	121,801.9415	122,150.9126	2219.6	21	PPSO [50]	125,503.09	127,886.04	129,631.35	6237.6	40
Jaya-SML [19]	121,476.3977	121,689.0773	122,039.8731	2210.9	20	SSA [50]	123,565.75	125,408.20	127,442.23	4300.2	39
MABC [51]	121,412.59	121,493.19	121,431.58	2147.1	8	MPA [50]	123,180.98	124,750.53	126,614.40	3915.5	38
SGO [13]	121,509.82092	122,025.1179	123,527.6187	2244.316	22	PPSO [48]	121,412.54	121,413.95	121,412.59	2147.0	6
MSGO [13]	121,426.70390	121,656.9571	122,048.2807	2161.1987	16	HBB-BC [52]	121,471.72	122,137.42	121,984.24	2206.2	19
PARPSO [53]	122,256.30	–	122,634.00	2990.8	36	CSA [54]	121,425.61	–	–	2160.1	15
ESCSDO10 [15]	121,626.97	122,351.7	123,128.9	2361.5	27	PSO-CGSA [55]	121,655.40100	122,324.58390	–	2389.896	28
HGS [15]	121,869.5	122,751.2	124,042.9	2604.0	32	KHA-IV [56]	121,412.60	121,415.00	121,413.15	2147.1	9
SDO [15]	121,750.2	122,460.1	123,222.7	2484.7	31	IODPSO-G [57]	121,414.93	121,426.42	121,416.54	2149.4	12
ESNS [6]	121,415.6	121,742.2	122,155.2	2150.1	13	IODPSO-L [57]	121,420.98	121,431.62	121,424.62	2155.5	14
SNS [6]	121,465.7	121,640.8	122,267.9	2200.2	18	C-GRASP-SaDE [58]	121,414.62	122,245.69	121,736.03	2149.1	10
GBO [6]	122,103.7	122,859.7	124,411.4	2838.2	35	CBA [20]	121,412.55	121,436.15	121,418.98	2147.0	7

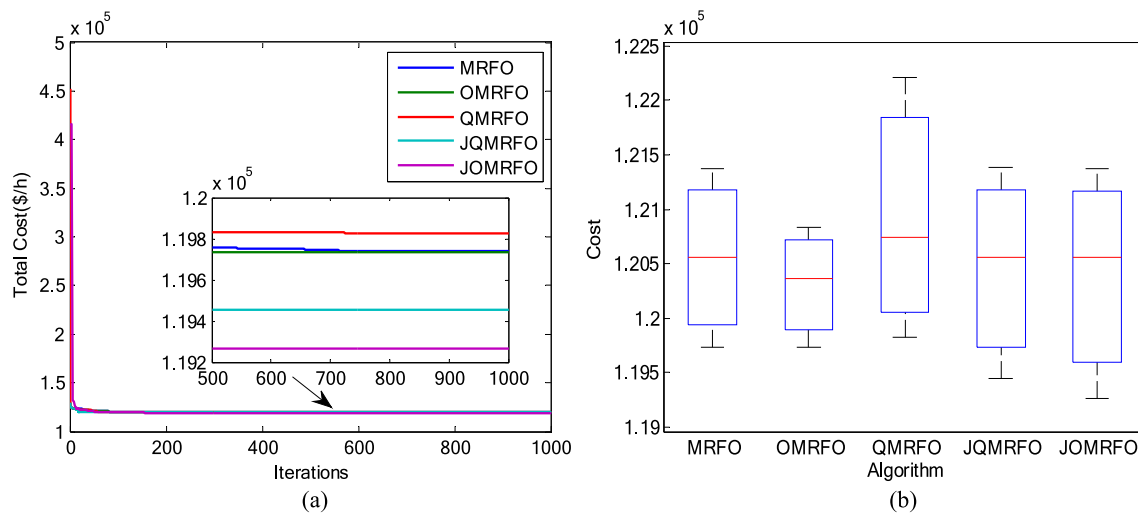


Fig. 13 Convergence characteristics and boxplot of test case 3

Table 9 Comparison of the statistical results of different methods for test case 4 (50-run)

Algorithm	Best cost	Mean cost	Worst cost	StD	Cost savings (\$/h)	Rank	Time (s.)
MRFO	1,559,900.00	1,559,905.65	1,559,961.5	17.00	10,591.254	5	45.27
OMRFO	1,555,700.00	1,555,703.17	1,555,758.8	12.62	6391.254	4	49.99
QMRFO	1,550,885.57	1,550,885.838	1,550,886.84	0.4633	1576.8243	3	51.83
JQMRFO	1,550,061.4190	1,552,865.3973	1,556,599.164	214.343	752.67329	2	51.49
JOMRFO	1,549,308.7457	1,549,308.74572	1,549,308.7460	8.62439e-005	0.0	1	49.96
OIWO [9]	1,559,405.45	1,559,405.61	1,559,405.88	–	10,096.7043	6	46.80
KHA [11]	1,560,173.88	1,560,176.74	1,560,177.80	–	10,865.134	11	–
OKHA [11]	1,560,146.95	1,560,148.92	1,560,149.97	–	10,838.2043	10	–
SSA [21]	1,658,384.25	1,658,384.88	1,658,386.57	0.2	109,075.50429	23	50.42
SDE [16]	1,560,236.85	–	–	–	10,928.1043	12	–
OGWO [61]	1,559,709.97	1,559,713.26	1,559,743.47	–	10,401.2243	7	41.77
GWO [61]	1,559,953.18	1,560,132.93	1,560,228.40	–	10,644.4343	9	45.51
HHO [16]	1,562,796.85	1,565,182.21	–	–	13,488.1043	17	–
HHO-AβHC [16]	1,559,748.49	1,559,755.53	–	–	10,439.7443	8	86,462.9
SSA [62]	1,579,734.50	1,595,468.41	–	–	30,425.7543	18	–
HSSA [62]	1,562,639.94	1,563,982.61	–	–	13,331.1943	16	–
EO [38]	1,653,800	1,654,783.3	1,654,850	–	104,491.25429	19	–
EMA-SS [63]	1,657,013.3376	1,657,015.4612	1,657,021.1009	–	107,704.59189	20	–
EMA [63]	1,657,013.3384	1,657,019.1354	1,657,043.5078	–	107,704.59269	21	–
MTVPSO [60]	1,560,436.71	1,560,446.22	1,560,460.55	0.00003	11,127.96429	15	–
DHS [59]	1,657,944.8622	1,657,944.8627	1,657,944.8652	0.0005	108,636.11650	22	4.6
aPSO [59]	1,560,435.88	1,560,444.56	1,560,461.52	0.0000076	11,128.13429	14	5.98
haDEPSO [59]	1,560,434.54	1,560,440.32	1,560,460.89	0.0000003	11,128.0042	13	3.25

Table 10 Best solution for test case 4 obtained by MRFO and JOMRFO

Unit	MRFO	JOMRFO	Unit	MRFO	JOMRFO	Unit	MRFO	JOMRFO	Unit	MRFO	JOMRFO
1	106.8	98.11041	36	499.7	499.9669	71	139.1	231.406	106	953.8	953.901
2	188.7	185.7503	37	240.9	240.9105	72	199.8	262.6707	107	951.9	951.9978
3	189.8	189.959	38	158.0	240.7703	73	210.8	207.1355	108	1005.8	1005.992
4	181.6	189.7303	39	774.0	770.8742	74	177.7	260.7744	109	1012.7	1012.985
5	129.3	129.2846	40	769.0	768.8646	75	375.1	309.4526	110	1021.0	1020.917
6	188.3	189.2304	41	3.5	3.12493	76	180.2	176.9866	111	1015.0	1014.963
7	477.3	489.9706	42	3.1	3.133451	77	237.6	181.8234	112	94.2	94.04321
8	490.0	489.9353	43	193.1	244.3003	78	334.6	331.2487	113	94.9	94.10246
9	495.1	495.9788	44	185.1	238.3876	79	530.6	530.4266	114	94.0	94.05995
10	488.5	495.6918	45	177.4	222.4717	80	530.9	530.7371	115	249.3	244.7546
11	495.9	495.894	46	233.0	248.1506	81	278.1	228.1761	116	272.5	244.18
12	493.9	495.9898	47	230.4	193.8849	82	56	56.09643	117	245.3	258.1809
13	506.0	505.9581	48	233.4	248.8909	83	115.5	115.2324	118	95.7	108.5646
14	509.0	508.0624	49	244.1	239.76	84	115.7	115.0142	119	95.7	95.02639
15	473.8	505.9481	50	249.2	246.9251	85	142.4	115.4671	120	116.6	117.3701
16	505.0	504.8511	51	243.5	165.1673	86	207.5	207.2518	121	175.7	176.9341
17	506.0	505.9254	52	238.3	238.2055	87	216.7	207.0329	122	12.2	2.717836
18	505.7	505.9952	53	166.7	165.3106	88	175.8	185.2649	123	4.1	4.078055
19	504.8	504.7425	54	166.1	184.7256	89	180.0	176.3112	124	16.3	15.24162
20	504.8	504.9936	55	180.6	180.1874	90	177.0	182.3811	125	20.7	9.143533
21	504.5	504.8531	56	180.1	181.2729	91	178.9	175.4894	126	12.7	21.40333
22	504.8	504.9739	57	103.7	123.635	92	579.8	579.9112	127	10.2	10.1317
23	504.5	504.8993	58	199.7	199.0353	93	644.8	643.9717	128	112.8	112.6642
24	504.9	504.9341	59	311.3	259.3577	94	983.7	983.5853	129	4.1	4.158644
25	535.4	536.1035	60	296.5	258.8036	95	977.5	977.6861	130	11.1	5.034401
26	536.9	536.7732	61	168.2	163.1203	96	682.0	681.991	131	10.5	5.11919
27	548.6	548.8605	62	95.4	95.24854	97	719.8	719.9542	132	50.2	50.6826
28	548.9	548.8007	63	186.6	162.3826	98	717.8	717.9645	133	5.2	5.107745
29	500.3	500.6655	64	296.9	168.3336	99	719.8	719.9288	134	42.2	42.03168
30	500.9	500.9549	65	373.9	466.0522	100	963.6	963.9397	135	42.0	42.12942
31	505.8	505.7274	66	257.0	209.5857	101	956.4	957.9859	136	41.5	41.15844
32	505.7	504.977	67	486.9	474.6618	102	1007.0	1006.972	137	17.2	17.02752
33	506.0	505.9838	68	482.8	470.9641	103	1005.9	1005.806	138	9.1	7.192504
34	505.9	505.9297	69	130.8	132.1071	104	1012.9	1012.911	139	11.4	7.234938
35	488.0	498.9722	70	234.6	234.8118	105	1019.9	1019.99	140	28.6	23.99917
Min cost (\$/h)										1,559,900.0	1,549,308.7457

test system (10-unit) to establish reasonable ranges for N and T_{max} . Preliminary findings established initial ranges for N (50–90) and T_{max} (100–300) based on observations of premature convergence for smaller N values and high computational cost for larger N and T_{max} values without significantly improving fuel cost. As system size increased, reflecting a rise in system complexity and search space dimensionality, the parameter ranges were expanded to allow for more thorough exploration. Specifically, for the 140-unit system, the N range was extended to 120, and T_{max} was increased to 6000 to account for the greater search effort required. These adjustments ensured sufficient exploration without incurring excessive computational overhead. A factorial design was used to systematically evaluate combinations of N and T_{max} within these ranges for each system size. To ensure statistical reliability, multiple independent runs were conducted—20 runs for the 10, 15, and 40-unit systems and 50 runs

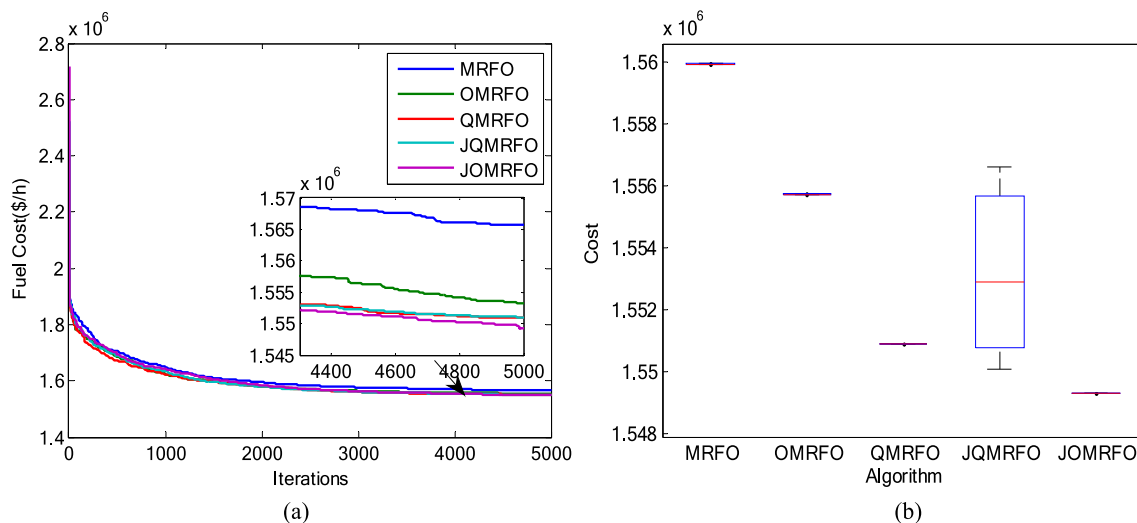


Fig. 14 Convergence characteristics and boxplot of test case 4

Table 11 Comparison of the statistical results of different methods for test case 5 (50-run)

Algorithm	Best cost	Mean cost	Worst cost	StD	Cost savings (\$/h)	Rank	Time (s.)
MRFO	9981.7620	9983.0185	9984.4032	0.711765	0.38499	5	51.90
OMRFO	9981.4866	9983.6764	10,020.572	5.3309	0.10959	4	54.56
QMRFO	9981.4761	9982.8826	9985.3664	0.80347	0.0991	3	55.44
JQMRFO	9981.4483	9982.8826	9985.3663	0.80347	0.0713	2	55.60
JOMRFO	9981.37701	9982.5964	9984.30708	0.69054	0.0	1	54.00
OIWO [9]	9981.9834	9982.991	9983.998	—	0.6064	6	17.3
ED-DE [59]	10,012.68	—	—	—	31.3030	10	—
CGA-MU [59]	10,143.73	—	—	—	162.3530	12	—
IGA-MU [59]	10,042.47	—	—	—	61.0930	11	—
ORCCRO [65]	10,004.20	10,004.21	10,004.45	—	22.8230	7	19
BBO [66]	10,008.71	10,009.16	10,010.59	—	27.3330	9	—
DE/BBO [66]	10,007.05	10,007.56	10,010.25	—	25.6730	8	—

for the 140 and 160-unit systems. Performance was primarily evaluated based on the minimum fuel cost achieved, while robustness (standard deviation of results) and computational cost (average execution time) were considered secondary metrics. The results, summarized in Table 13, show that the optimal parameter settings effectively balanced these metrics. For instance, in the 10-unit system, increasing N beyond 75 slightly improved robustness but did not significantly reduce fuel cost, while computational cost increased. Therefore, $N = 75$ was selected, as detailed in Table 14. Similarly, for larger systems, the expanded ranges ensured sufficient search capability without incurring excessive computational overhead. This systematic parameter tuning process ensured consistent and well-justified settings across all system sizes, enabling reliable and fair performance comparisons between the algorithm variants.

Table 12 Best solution for test case 5 obtained by JOMRFO

Unit	Fuel	Output power (MW)	Unit	Fuel	Output power (MW)	Unit	Fuel	Output power (MW)	Unit	Fuel	Output power (MW)	Unit	Fuel	Output power (MW)
1	2	218.5940	33	1	279.6489	65	1	279.9697	97	1	287.7284	129	3	426.1381
2	1	212.7020	34	3	239.6394	66	3	240.1769	98	3	239.5051	130	1	275.8975
3	1	279.6489	35	1	279.9697	67	1	287.7284	99	3	426.1381	131	2	218.5940
4	3	239.6394	36	3	240.1769	68	3	239.5051	100	1	275.8975	132	1	212.7020
5	1	279.9697	37	1	287.7284	69	3	426.1381	101	2	218.5940	133	1	279.6489
6	3	240.1769	38	3	239.5051	70	1	275.8975	102	1	212.7020	134	3	239.6394
7	1	287.7284	39	3	426.1381	71	2	218.5940	103	1	279.6489	135	1	279.9697
8	3	239.5051	40	1	275.8975	72	1	212.7020	104	3	239.6394	136	3	240.1769
9	3	426.1381	41	2	218.5940	73	1	279.6489	105	1	279.9697	137	1	287.7284
10	1	275.8975	42	1	212.7020	74	3	239.6394	106	3	240.1769	138	3	239.5051
11	2	218.5940	43	1	279.6489	75	1	279.9697	107	1	287.7284	139	3	426.1381
12	1	212.7020	44	3	239.6394	76	3	240.1769	108	3	239.5051	140	1	275.8975
13	1	279.6489	45	1	279.9697	77	1	287.7284	109	3	426.1381	141	2	218.5940
14	3	239.6394	46	3	240.1769	78	3	239.5051	110	1	275.8975	142	1	212.7020
15	1	279.9697	47	1	287.7284	79	3	426.1381	111	2	218.5940	143	1	279.6489
16	3	240.1769	48	3	239.5051	80	1	275.8975	112	1	212.7020	144	3	239.6394
17	1	287.7284	49	3	426.1381	81	2	218.5940	113	1	279.6489	145	1	279.9697
18	3	239.5051	50	1	275.8975	82	1	212.7020	114	3	239.6394	146	3	240.1769
19	3	426.1381	51	2	218.5940	83	1	279.6489	115	1	279.9697	147	1	287.7284
20	1	275.8975	52	1	212.7020	84	3	239.6394	116	3	240.1769	148	3	239.5051
21	2	218.5940	53	1	279.6489	85	1	279.9697	117	1	287.7284	149	3	426.1381
22	1	212.7020	54	3	239.6394	86	3	240.1769	118	3	239.5051	150	1	275.8975
23	1	279.6489	55	1	279.9697	87	1	287.7284	119	3	426.1381	151	2	218.5940
24	3	239.6394	56	3	240.1769	88	3	239.5051	120	1	275.8975	152	1	212.7020
25	1	279.9697	57	1	287.7284	89	3	426.1381	121	2	218.5940	153	1	279.6489
26	3	240.1769	58	3	239.5051	90	1	275.8975	122	1	212.7020	154	3	239.6394
27	1	287.7284	59	3	426.1381	91	2	218.5940	123	1	279.6489	155	1	279.9697
28	3	239.5051	60	1	275.8975	92	1	212.7020	124	3	239.6394	156	3	240.1769
29	3	426.1381	61	2	218.5940	93	1	279.6489	125	1	279.9697	157	1	287.7284
30	1	275.8975	62	1	212.7020	94	3	239.6394	126	3	240.1769	158	3	239.5051
31	2	218.5940	63	1	279.6489	95	1	279.9697	127	1	287.7284	159	3	426.1381
32	1	212.7020	64	3	239.6394	96	3	240.1769	128	3	239.5051	160	1	275.8975
Min cost (\$/h)													9981.37701	

5.4 Comprehensive performance analysis

5.4.1 Capability to escape local optima

An algorithm's ability to escape local optima depends on maintaining population diversity and identifying high-potential regions for exploration. The results presented in Tables 2, 5, 7, and 10, supported by the statistical analyses in Tables 3, 6, 8, 9, and 11, confirm the enhanced algorithms' effectiveness in addressing this challenge. The convergence patterns illustrated in Figs. 11a, 12a, 13a, 14a and 15a demonstrate that the enhanced algorithms achieve faster convergence and higher solution quality compared to the standard MRFO, which often stagnates in local optima and fails to converge to the optimum solution in most test cases. Notably, JOMRFO exhibits the most efficient and rapid progression toward optimal solutions, as evidenced by its convergence behaviour. The

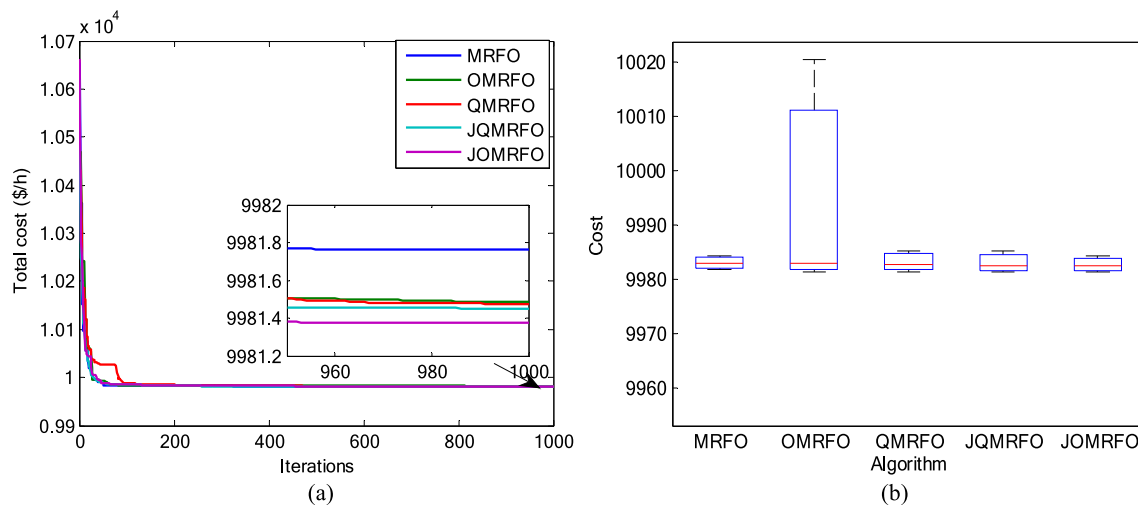


Fig. 15 Convergence characteristics and boxplot of test case 5

Table 13 Optimal parameter settings of the 5 test systems

Test system	N range	T_{max} range	N	T_{max}	System description
10-unit test system	50–90	100–300	75	200	Small-scale test system VPL effects and power losses
15-unit test system	50–100	800–1200	70	1000	Small-scale test system with POZs and power losses
40-unit test system	50–100	800–1200	60	1000	Medium-scale test system with VPL effects
140-unit test system	50–120	1000–6000	105	5000	Large-scale test system with VPL effects
160-unit test system	50–120	800–1200	85	1000	Large-scale test system with MFO and VPL formed by duplicating a basic 10-unit test system

algorithm consistently achieves lower fuel costs at an accelerated rate compared to the standard MRFO and other variants, reflecting its effective balance between exploration and exploitation. This enhanced performance is primarily attributed to the mechanisms of OBL and the jumping rate, as described earlier. Together, these mechanisms ensure sufficient exploration while enabling focused exploitation in promising regions, allowing the algorithm to overcome stagnation and locate optimal solutions efficiently.

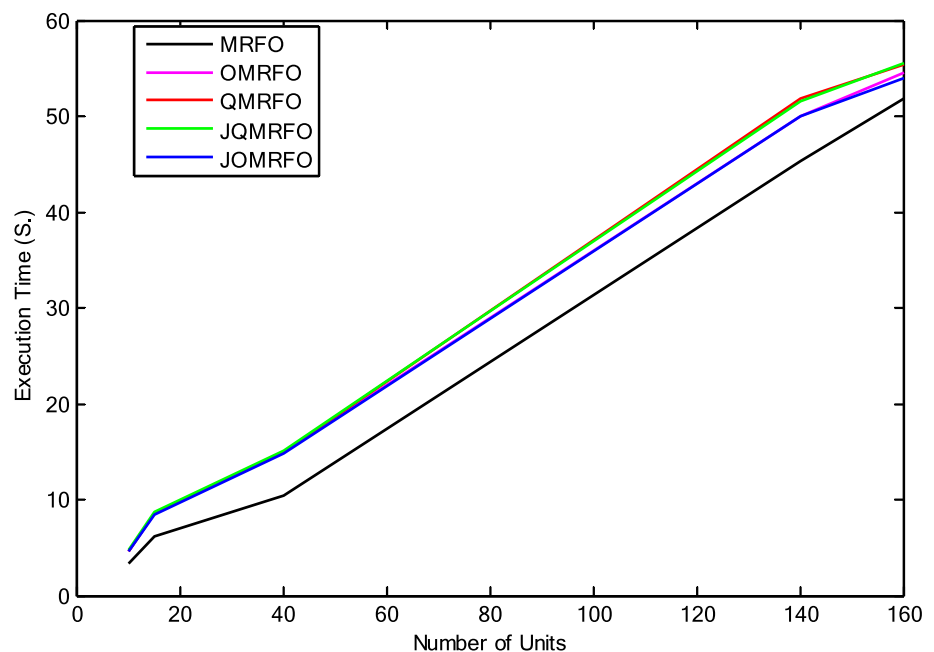
5.4.2 Scalability analysis

The scalability analysis evaluates the execution times of MRFO and its enhanced variants across various system sizes (10, 15, 40, 140, and 160 units). While all algorithms share the same asymptotic computational complexity ($O(T_{max}Nd)$), their practical execution times differ due to the additional computational overhead introduced by opposition-based techniques and generation jumping mechanisms in the enhanced variants. As shown in Fig. 16, MRFO demonstrates the best scalability, with execution times increasing from 3.29 s for the 10-unit test system to 51.90 s for the 160-unit system, reflecting a near-linear growth. The enhanced variants, such as JOMRFO, show slightly higher execution times (e.g., 4.56 s for the 10-unit test system and 54.0 s for the 160-unit test system) but achieve improved optimization results, as evidenced by their faster convergence rates and lower total costs (Figs. 11a, 12a, 13a, 14a, 15a). These results highlight the trade-off between runtime efficiency and solution quality. While MRFO offers superior scalability for speed-sensitive scenarios, the enhanced variants provide better optimization outcomes at a slightly higher computational demand.

The observed linear growth in execution time is expected to continue with larger configurations. For example, MRFO's execution time is predicted to reach approximately 58–60 s for a 200-unit system and around 65–70 s

Table 14 Effect of changing N on 10-unit test system (200 iterations)

N	Time (s.)	Best Cost-\$/h	Mean Cost-\$/h	Max Cost -\$/h	StD
40	1.15	111,830.4813	111,866.8763	111,910.3027	22.1400
45	1.75	111,827.6502	111,857.8200	111,922.7219	24.0016
50	1.93	111,825.0487	111,850.6494	111,889.9647	18.1999
55	2.16	111,832.3742	111,855.4196	111,919.8774	20.4588
60	2.42	111,827.1605	111,845.0346	111,868.9389	13.2366
65	2.79	111,825.6526	111,846.0572	111,880.2234	15.1684
70	3.01	111,827.3043	111,844.7105	111,870.7611	13.731
75	3.29	111,823.2656	111,846.1314	111,875.4325	13.444
80	3.42	111,826.5087	111,844.1096	111,890.4064	15.915
85	3.74	111,830.5701	111,844.4379	111,882.3510	12.75
90	3.89	111,824.0295	111,840.0816	111,857.7568	9.77
100	4.21	111,824.9997	111,837.0826	111,865.29409	9.39267

Fig. 16 Scalability trends of MRFO and its enhanced variants across different system sizes


for a 300-unit system. At the same time, the enhanced variants are anticipated to exhibit slightly higher times due to their added computational mechanisms. Despite this, the enhanced variants maintain their relative performance advantages by avoiding local optima and converging to better solutions. These findings suggest that MRFO and its enhanced variants can efficiently handle larger problem sizes, making them scalable for practical applications.

5.4.3 Solution quality

Tables 2, 5, and 7 present the best fuel cost obtained by MRFO and the four proposed enhanced algorithms for test cases 1, 2, and 3. Table 10 presents the best cost achieved by MRFO and JOMRFO for test case 4, and Table 12 presents the best cost achieved by JOMRFO for test case 5. These costs are compared to the results obtained by the recently published algorithms, as shown in Tables 2, 3, 5, 6, 7, 8, 9, and 11. The results clearly demonstrate that the performance of the proposed algorithms is better in terms of the quality of solutions obtained compared to other reported algorithms. Comparing the results of the MRFO, OMRFO, QMRFO, JQMRFO, and JOMRFO algorithms, it is found that the JOMRFO has a superior performance compared to the other methods by

achieving the best results in test cases 1, 3, 4, and 5, and almost the same optimal solution (32,575.54275 \$/h) achieved by JQMRFO (32,575.54273 \$/h) in test case 2.

5.4.4 Robustness

To evaluate an algorithm's robustness accurately, multiple runs are necessary. The proposed algorithms were executed 20 times in this study for test cases 1, 2, and 3, and 50 times for test cases 4 and 5. The statistical findings of these runs are detailed in Tables 3, 6, 8, 9, and 11. As evidenced by these tables, the proposed algorithms demonstrate superior statistical indices compared to other methods, indicating their effectiveness and efficiency. Tables 3, 9, and 11 demonstrate JOMRFO's superior performance, achieving the best, mean, and worst cost metrics compared to other algorithms. Moreover, Table 8 shows that JOMRFO outperforms other algorithms as it obtains better values of best and worst costs. However, OMRFO achieves the best value of mean cost. Table 6 indicate that the JQMRFO outperforms other algorithms by achieving the best cost. Furthermore, analysis of the boxplots in Figs. 11b, 13b, 14b, and 15b shows that the JOMRFO algorithm outperforms the other algorithms in terms of performance and consistency. These results confirm that the proposed JOMRFO is the best algorithm for solution quality and robustness compared to other proposed algorithms for solving ED problems in the considered test systems.

6 Conclusion and future work

In this paper, four effective enhanced variants of the Manta Ray Foraging Optimization Algorithm (MRFO) have been proposed. These variants are based on using opposition-based learning (OBL), quasi-oppositional learning (QOBL), and the jumping rate (J_r) to improve the performance of the standard MRFO algorithm. This study demonstrates that incorporating these enhancements into MRFO significantly improves solution quality and convergence speed, offering promising solutions for real-world power generation optimization. Key findings are summarized as follows:

1. The enhanced MRFO variants improve algorithm performance by addressing local optima issues and enhancing exploration while maintaining the same asymptotic computational complexity and space efficiency as the original MRFO. These features ensure a balance of memory efficiency and computational feasibility.
2. The proposed algorithms have provided better results compared to the conventional MRFO algorithm in the five considered test cases. Furthermore, the results obtained by MRFO are superior to those obtained by the other reported algorithms in all cases, demonstrating its effectiveness.
3. Comparing the simulation and statistical results obtained by OMRFO, QMRFO, JQMRFO, and JOMRFO. It has been found that the JOMRFO outperforms other methods in terms of stability and reliability, suggesting that the combination of opposition-based learning (OBL) and a jumping rate (J_r) effectively directs the search process, which results in a more robust and reliable exploration of the solution space.
4. Using JOMRFO to solve non-convex economic dispatch (ED) problems resulted in significant annual cost savings of up to 3,730,343.8584 \$, 54,641,376 \$, 955,501,417.5804 \$, and 1,422,212.28 \$ for test cases 1, 3, 4, and 5. Additionally, implementing JQMRFO for test case 2 results in annual cost savings of up to 2,998,787.148 \$.
5. The convergence graphs demonstrably show that the enhanced variants achieve faster convergence and better accuracy than standard MRFO. This improved efficacy translates to reduced computational time and enhanced decision-making capabilities in real-time power system operations.

In conclusion, this study demonstrates the JOMRFO algorithm's potential to significantly impact power generation optimization in modern power systems. Its scalability, effectiveness, and superior performance make it a valuable tool for tackling the complex challenges of large-scale power systems worldwide.

Future research should investigate the applicability of JOMRFO to larger, dynamic economic dispatch systems, incorporating constraints on thermal power units to enhance operational reliability and stability. Additionally, exploring its potential in microgrid energy management, particularly in integrating renewable energy sources, offers a promising path towards sustainable energy practices and addressing global energy challenges related to climate change and resource depletion.

Authors' contributions All the work related to this manuscript is done by single author S. R. Spea. S. R. Spea: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing—Original Draft, Writing—Review & Editing, Visualization, Supervision.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB).

Availability of data and materials Utilizing data sourced from the referenced literature [11, 13, 15, 41].

Declarations

Conflict of interests The authors have no competing interests to declare.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Hussien A, Kamel S, Ebeed M, Yu J (2021) A developed approach to solve economic and emission dispatch problems based on moth-flame algorithm. *Electr Power Compon Syst* 49(1):94–107
2. Singh N et al (2023) Novel heuristic optimization technique to solve economic load dispatch and economic emission load dispatch problems. *Electronics* 12(13):2921
3. Spea SR (2023) Social network search algorithm for combined heat and power economic dispatch. *Electr Power Syst Res* 221:109400
4. Marzbani F, Abdelfatah A (2024) Economic dispatch optimization strategies and problem formulation: a comprehensive review. *Energies*. <https://doi.org/10.3390/en17030550>
5. Hassan MH et al (2024) Efficient economic operation based on load dispatch of power systems using a leader white shark optimization algorithm. *Neural Comput Appl* 36:10613–10635
6. Hassan MH, Kamel S, Jurado F, Desideri U (2024) Global optimization of economic load dispatch in large scale power systems using an enhanced social network search algorithm. *Int J Electr Power Energy Syst* 156:109719
7. Spea SR (2024) Optimizing economic dispatch problems in power systems using manta ray foraging algorithm: an oppositional-based approach. *Comput Electr Eng* 117:109279
8. Lakshmi VSG, Vanithasri M, Rao MVG (2024) An efficient solution for economic load dispatch using cheetah optimizer with opposition-based learning and adaptive weighting factor. *Int J Intell Syst* 17(3):139–148
9. Barisal AK, Prusty RC (2015) Large scale economic dispatch of power systems using oppositional invasive weed optimization. *Appl Soft Comput* 29:122–137
10. Basetti V et al (2021) Economic emission load dispatch problem with valve-point loading using a novel quasi-oppositional-based political optimizer. *Electronics* 10:2596
11. Bulbul SMA, Pradhan M, Roy PK, Pal T (2018) Opposition-based krill herd algorithm applied to economic load dispatch problem. *Ain Shams Eng J* 9:423–440
12. Das D, Bhattacharya A, Ray RN (2018) Quasi-oppositional grey wolf optimizer algorithm for economic dispatch. *Ind J Sci Technol*. <https://doi.org/10.17485/ijst/2018/v11i41/108579>

13. Secui DC et al (2024) Modified social group optimization to solve the problem of solve the problem of economic emission dispatch with the incorporation of wind power. *Sustainability*. <https://doi.org/10.3390/su16010397>
14. Hoque S et al (2023) Generalized normal distribution optimization algorithm for economic dispatch with renewable resources integration. *JEPT* 5(3):1–19
15. Hassan MH et al (2023) A developed eagle-strategy supply-demand optimizer for solving economic load dispatch problems. *Ain Shams Eng J*. <https://doi.org/10.1016/j.asej.2022.102083>
16. Al-Betar MA et al (2023) A hybrid Harris Hawks optimizer for economic load dispatch problems. *Alex Eng J* 64:365–389
17. Al-Betar MA, Awadallah MA, Zitar RA, Assaleh Kh (2023) Economic load dispatch using memetic sine cosine algorithm. *J Ambient Intell Humaniz Comput* 14:11685–11713
18. Yu J, Kim C-H, Rhee S-B (2020) Clustering cuckoo search optimization for economic load dispatch problem. *Neural Comput Appl* 32(22):16951–16969
19. Yu J, Kim C-H, Wadood A, Khurshaid T, Rhee S-B (2019) Jaya algorithm with self-adaptive multi-population and Lévy flights for solving economic load dispatch problems. *IEEE Access* 7:21372–21384
20. Adarsh BR, Raghunathan T, Jayabarathi T, Yang XS (2016) Economic dispatch using chaotic bat algorithm. *Energy* 96:666–675
21. Bhattacharjee K, Patel N (2022) A comparative study of economic load dispatch with complex non- linear constraints using salp swarm algorithm. *Sci Iran D* 29(2):676–692
22. Zhao W, Zhang Zh, Li W (2020) Manta ray foraging optimization: an effective bio-inspired optimizer for engineering applications. *Eng Appl Artif Intell*. <https://doi.org/10.1016/j.engappai.2019.103300>
23. Izci D, Ekinci S, Eker E, Kayri M (2020) Improved manta ray foraging optimization using opposition-based learning for optimization problems. In: *International congress on human–computer interaction, optimization and robotic applications (HORA)*, 1–6, IEEE, Ankara, Turkey
24. Zhang X-Y et al (2023) Manta ray foraging optimization algorithm with mathematical spiral foraging strategies for solving economic load dispatching problems in power systems. *Alex Eng J* 70:613–640
25. Zhu F, Wang W, Li S (2022) Application of improved manta ray foraging optimization algorithm in coverage optimization of wireless sensor networks. *Comput Intell Neurosci*. <https://doi.org/10.1155/2022/3082933>
26. Almutairi SZ, Mohamed EA, El-Sousy FFM (2023) A novel adaptive manta-ray foraging optimization for stochastic ORPD considering uncertainties of wind power and load demand. *Mathematics* 11:2591
27. Fragaa ES, Yanga L, Papageorgiou LG (2012) On the modelling of valve point loadings for power electricity dispatch. *Appl Energy* 91(1):301–303
28. Boucekara HREH, Chaib AE, Abido MA (2018) Optimal power flow using GA with a new multi-parent crossover considering: prohibited zones, valve-point effect, multi-fuels and emission. *Electr Eng* 100:151–165
29. Nguyen TT, Vo DN, Quynh NV, Dai LV (2018) Modified cuckoo search algorithm: a novel method to minimize the fuel cost. *Energies* 11:1328
30. Tizhoosh HR (2005) Opposition-based learning: a new scheme for machine intelligence. *CIMCA* 695–701.
31. Rahnamayan S, Tizhoosh HR, Salama MMA (2007c) Quasi-oppositional differential evolution. *IEEE CEC* 2229–2236.
32. Salaria UA, Menhas MI, Manzoor S (2021) Quasi oppositional population based global particle swarm optimizer with inertial weights (QGPSO-W) for solving economic load dispatch problem. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2021.3116066>
33. Yu X, Xu WY, Li CL (2021) Opposition-based learning grey wolf optimizer for global optimization. *Knowl Based Syst (KBS)* 226:107139
34. Ciornei I, Kyriakides E (2012) A GA-API solution for the economic dispatch of generation in power system operation. *IEEE Trans Power Syst* 27(1):233–242
35. Chaudhary V, Dubey HM, Pandit M, Salkuti SR (2023) A chaotic Jaya algorithm for environmental economic dispatch incorporating wind and solar power. *AIMS Energy* 12(1):1–30
36. Hassan MH, Kamel S, Abualigah L, Eid A (2021) Development and application of slime mould algorithm for optimal economic emission dispatch. *Expert Syst Appl* 182:115205
37. Atre A, Agnihotri S, Verma HK (2020) Hybrid EO-SCA Based Economic Load Dispatch. In: *2020 IEEE first international conference on smart technologies for power, energy and control (STPEC)*
38. Agnihotri S, Atre A, Verma HK (2020) Equilibrium optimizer for solving economic dispatch problem. *IEEE 9th Power India International Conference (PIICON)*.
39. Yang X-S, Hosseini SSS, Gandomi AH (2012) Firefly algorithm for solving non-convex economic dispatch problems with valve loading effect. *Appl Soft Comput* 12(3):1180–1186
40. Ghorbani N, Vakili S, Babaei E, Sakhavati A (2014) Particle swarm optimization with smart inertia factor for solving nonconvex economic load dispatch problems. *Int Trans Elect Energy Syst* 24:1773–1781
41. Abdullah MN, Abu Bakar AH, Rahim NA, Moklis H (2013) Economic load dispatch with nonsmooth cost functions using evolutionary particle swarm optimization. *IEEJ Trans Electr Electron Eng* 8(S1):S30–S37
42. Aragón VS, Esquivel SC, Coello CA (2015) An immune algorithm with power redistribution for solving economic dispatch problems. *Inf Sci* 295:609–632

43. Jadhav HT, Roy R (2013) Gbest guided artificial bee colony algorithm for environmental/economic dispatch considering wind power. *Expert Syst Appl* 40(16):6385–6399
44. Ghorbani N, Babaei E (2016) Exchange market algorithm for economic load dispatch. *Int J Electr Power Energy Syst* 75:19–27
45. Dey SHN (2014) Teaching learning based optimization for different economic dispatch problems. *Sci Iran* 21(3):870–884
46. Barati H, Sadeghi M (2018) An efficient hybrid MPSO-GA algorithm for solving nonsmooth/non-convex economic dispatch problem with practical constraints. *Ain Shams Eng J* 9(4):1279–1287
47. Khamsawang S, Jirawibhakorn S (2010) DSPSO-TSA for economic dispatch problem with nonsmooth and noncontinuous cost functions. *Energy Convers Manag* 51(2):365–375
48. Gholamghasemi M et al (2019) A new solution to the non-convex economic load dispatch problems using phasor particle swarm optimization. *Appl Soft Comput* 79:111–124
49. Hassan MH, Kamel S, Jurado F, Ebeed M, Elnaggar MF (2023) Economic load dispatch solution of large-scale power systems using an enhanced beluga whale optimizer. *Alex Eng J* 72:573–591
50. Pan J, Shan J, Chu S, Jiang S, Zheng S, Liao L (2022) A multigroup marine predator algorithm and its application for the power system economic load dispatch. *Energy Sci Eng* 10:1840–1854
51. Secui DC (2015) A new modified artificial bee colony algorithm for the economic dispatch problem. *Energy Convers Manag* 89:43–62
52. Labbi Y, Attous DB (2017) A hybrid big bang–big crunch optimization algorithm for solving the different economic load dispatch problems. *Int J Syst Assur Eng Manag* 8:275–286
53. Baek M-K, Park J-B, Lee KY (2016) An improved attractive and repulsive particle swarm optimization for nonconvex economic dispatch problems. *IFAC-PapersOnLine* 49(27):284–289
54. Basu M, Chowdhury A (2013) Cuckoo search algorithm for economic dispatch. *Energy* 60:99–108
55. Gajić M, Arsić S, Radosavljević J, Jevtić M, Perović B, Klimenta D, Milovanović M (2024) Behavior analysis of the new PSO-CGSA algorithm in solving the combined economic emission dispatch using non-parametric tests. *Appl Artif Intell*. <https://doi.org/10.1080/08839514.2024.2322335>
56. Mandal B, Roy PK, Mandal S (2014) Economic load dispatch using krill herd algorithm. *Int J Electr Power Energy Syst* 57:1–10
57. Qin Q, Cheng S, Zhang Q, Wei Y, Shi Y (2015) Multiple strategies based orthogonal design particle swarm optimizer for numerical optimization. *Comput Oper Res* 60:91–110
58. Neto JXV et al (2017) Solving non-smooth economic dispatch by a new combination of continuous GRASP algorithm and differential evolution. *Int J Electr Power Energy Syst* 84:13–24
59. Verma P, Parouha RP (2021) An advanced hybrid meta-heuristic algorithm for solving small- and large-scale engineering design optimization problems. *J Electr Syst Inf Technol* 8:10
60. Parouha RP (2019) Nonconvex/nonsmooth economic load dispatch using modified time-varying particle swarm optimization. *Comput Intell* 35(4):717–744
61. Pradhan M, Roy PK, Pal T (2018) Oppositional based grey wolf optimization algorithm for economic dispatch problem of power system. *Ain Shams Eng J* 9(4):2015–2025
62. Alkoffash MS et al (2021) A non-convex economic load dispatch using hybrid salp swarm algorithm. *Arab J Sci Eng* 46(9):8721–8740
63. Ghorbani N, Babaei E (2017) The exchange market algorithm with smart searching for solving economic dispatch problems. *Int J Manag Sci Eng Manag* 13:175–187. <https://doi.org/10.1080/17509653.2017.1365262>
64. Wang Y, Li B, Weise T (2010) Estimation of distribution and differential evolution cooperation for large scale economic load dispatch optimization of power systems. *Inf Sci* 180(12):2405–2420
65. Bhattacharjee K, Bhattacharya A, Dey SH (2014) Oppositional real coded chemical reaction optimization for different economic dispatch problems. *Int J Electr Power Energy Syst* 55:378–391
66. Bhattacharya A, Chattopadhyay PK (2010) Biogeography-based optimization for different economic load dispatch problems. *IEEE Trans Power Syst* 25(2):1064–1077

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

S. R. Spea¹

✉ S. R. Spea
shaimaa.sobaia@sh-eng.menofia.edu.eg

¹ Electrical Engineering Department, Faculty of Engineering, Menoufia University, Shibin-el-Kom, Egypt